V0.2.0

```
#   backend/common/topics.py
"""Shared LLM-security topic taxonomy (Core + Extended).
Only topic constants and pure helpers live here. No dependency on
intel_agent
and the collection console.
"""
from __future__ import annotations
import re
# Coverage / critic hard targets (full mode open_gaps).
CORE_SECURITY_TOPICS: list[str] = [
    "prompt injection",
    "jailbreak",
    "agent tool abuse",
    "data leakage",
    "model supply chain",
    "rag poisoning",
    "insecure output handling",
    "excessive agency",
    "model denial of service",
    "unbounded consumption",
    "model backdoor",
    "agent loop and cascading failure",
    "multi-agent communication attack",
    "unsafe code generation and execution",
    "sandbox escape",
    "plugin and mcp trust attack",
    "authentication and authorization bypass",
    "unsafe file handling",
]
# Searchable / labelable topics beyond Core (do not drive coverage quotas).
EXTENDED_SECURITY_TOPICS: list[str] = [
    "training and fine-tuning data poisoning",
    "adversarial examples",
    "model extraction",
    "membership inference",
    "model inversion",
    "training data extraction",
    "system prompt leakage",
    "context isolation failure",
    "memory poisoning",
    "agent impersonation",
    "server-side request forgery",
    "internal network reconnaissance",
    "misinformation and citation fabrication",
    "insufficient logging and monitoring",
    "model routing and downgrade attack",
]
ALL_SECURITY_TOPICS: list[str] = list(CORE_SECURITY_TOPICS) +
list(EXTENDED_SECURITY_TOPICS)
# Backward-compatible alias: historically meant "all searchable topics".
TARGET_SECURITY_TOPICS: list[str] = list(ALL_SECURITY_TOPICS)
TOPIC_KEYWORDS: dict[str, tuple[str, ...]] = {
```

```
    "prompt injection": (
        "prompt injection",
        "indirect prompt",
        "instruction injection",
        "prompt override",
    ),
    "jailbreak": (
        "jailbreak",
        "policy bypass",
        "guardrail bypass",
        "dan prompt",
        "safety bypass",
    ),
    "agent tool abuse": (
        "agent tool",
        "tool abuse",
        "plugin abuse",
        "function calling",
        "tool invocation attack",
        "confused deputy",
    ),
    "data leakage": (
        "data leakage",
        "secret exfiltration",
        "pii leak",
        "cross-tenant",
        "sensitive data exposure",
    ),
    "model supply chain": (
        "model supply chain",
        "model artifact",
        "unsafe model",
        "pickle",
        "model hub",
        "malicious model",
        "supply chain vulnerability",
    ),
    "rag poisoning": (
        "rag poisoning",
        "retrieval poisoning",
        "knowledge base poisoning",
        "vector database poison",
        "embedding poison",
    ),
    "insecure output handling": (
        "insecure output handling",
        "llm output injection",
        "xss via llm",
        "unsanitized model output",
        "output handling",
```

```
    ),
    "excessive agency": (
        "excessive agency",
        "over-permissioned agent",
        "unrestricted agent",
        "agent autonomy abuse",
    ),
    "model denial of service": (
        "model denial of service",
        "llm dos",
        "model dos",
        "resource exhaustion llm",
        "context flooding",
    ),
    "unbounded consumption": (
        "unbounded consumption",
        "token exhaustion",
        "runaway token",
        "cost amplification",
        "quota abuse llm",
    ),
    "model backdoor": (
        "model backdoor",
        "trojaned model",
        "backdoored llm",
        "sleeper agent model",
        "trigger phrase backdoor",
    ),
    "agent loop and cascading failure": (
        "agent loop",
        "cascading failure",
        "infinite agent loop",
        "recursive agent",
        "runaway agent",
    ),
    "multi-agent communication attack": (
        "multi-agent",
        "agent communication attack",
        "inter-agent",
        "agent message injection",
        "swarm attack",
    ),
    "unsafe code generation and execution": (
        "unsafe code generation",
        "code execution",
        "llm generated malware",
        "execute generated code",
        "eval generated code",
    ),
    "sandbox escape": (
```

```
        "sandbox escape",
        "container escape",
        "jail escape llm",
        "breakout sandbox",
    ),
    "plugin and mcp trust attack": (
        "mcp trust",
        "mcp attack",
        "plugin trust",
        "malicious mcp",
        "tool server compromise",
        "model context protocol",
    ),
    "authentication and authorization bypass": (
        "auth bypass",
        "authorization bypass",
        "authentication bypass",
        "privilege escalation llm",
        "broken access control llm",
    ),
    "unsafe file handling": (
        "unsafe file handling",
        "path traversal llm",
        "arbitrary file read",
        "file upload llm",
        "insecure file access",
    ),
    "training and fine-tuning data poisoning": (
        "training data poisoning",
        "fine-tuning poison",
        "finetune poison",
        "poisoned dataset",
        "data poisoning attack",
    ),
    "adversarial examples": (
        "adversarial example",
        "adversarial attack",
        "adversarial prompt",
        "input perturbation",
    ),
    "model extraction": (
        "model extraction",
        "model stealing",
        "functionality extraction",
        "api model theft",
    ),
    "membership inference": (
        "membership inference",
        "membership attack",
        "training set membership",
```

```
    ),
    "model inversion": (
        "model inversion",
        "inversion attack",
        "reconstruct training",
    ),
    "training data extraction": (
        "training data extraction",
        "memorization attack",
        "extract training data",
        "verbatim memorization",
    ),
    "system prompt leakage": (
        "system prompt leak",
        "system prompt leakage",
        "prompt disclosure",
        "reveal system prompt",
    ),
    "context isolation failure": (
        "context isolation",
        "cross-context leak",
        "session isolation failure",
        "tenant isolation llm",
    ),
    "memory poisoning": (
        "memory poisoning",
        "agent memory poison",
        "long-term memory attack",
        "memory injection",
    ),
    "agent impersonation": (
        "agent impersonation",
        "spoofed agent",
        "fake agent identity",
    ),
    "server-side request forgery": (
        "ssrf",
        "server-side request forgery",
        "llm ssrf",
    ),
    "internal network reconnaissance": (
        "internal network reconnaissance",
        "network recon llm",
        "internal scanning agent",
        "lateral movement llm",
    ),
    "misinformation and citation fabrication": (
        "citation fabrication",
        "hallucinated citation",
        "misinformation llm",
```

```
        "fabricated reference",
    ),
    "insufficient logging and monitoring": (
        "insufficient logging",
        "llm monitoring",
        "audit trail llm",
        "undetected llm abuse",
    ),
    "model routing and downgrade attack": (
        "model routing",
        "model downgrade",
        "router attack",
        "force weaker model",
    ),
}
TOPIC_ANCHORS: dict[str, list[str]] = {
    "prompt injection": [
        "An attacker embeds malicious instructions in content that a large
language model pr
ocesses, overriding the system prompt.",
        "Indirect prompt injection hides adversarial instructions in web
pages, documents, o
r tool outputs consumed by an LLM application.",
    ],
    "jailbreak": [
        "A jailbreak prompt bypasses the safety alignment of a large language
model to elici
t prohibited content.",
        "Adversarial prompting techniques defeat refusal behavior and
guardrails of aligned
chat models.",
    ],
    "agent tool abuse": [
        "An autonomous LLM agent is tricked into invoking dangerous tools such
as shell comm
ands or code execution.",
        "A confused-deputy attack abuses the permissions of an AI agent's
plugins, function
calling, or MCP servers.",
    ],
    "data leakage": [
        "A large language model leaks sensitive training data, secrets, or
personal informat
ion in its responses.",
        "Cross-tenant or session data exposure occurs in an LLM application,
revealing other
 users' prompts or documents.",
    ],
    "model supply chain": [
        "Malicious or tampered model artifacts execute code on load, for
example through uns
afe pickle deserialization.",
        "A compromised package or model hub distribution channel delivers
backdoored AI comp
onents.",
```

```
   ],
   "rag poisoning": [
      "An attacker poisons the document corpus or vector database of a
retrieval-augmented
 generation system to manipulate answers.",
      "Embedding or knowledge-base poisoning injects adversarial passages
that are retriev
```

```
ed and trusted by an LLM.",
    ],
    "insecure output handling": [
        "Downstream systems execute or render unsanitized LLM output, enabling
XSS, command
injection, or privilege abuse.",
        "Insecure output handling treats model text as trusted without
validation or encodin
g.",
    ],
    "excessive agency": [
        "An LLM agent is granted excessive permissions and takes high-impact
actions without
 adequate human oversight.",
        "Over-permissioned tool access lets an agent modify production systems
beyond the in
tended scope.",
    ],
    "model denial of service": [
        "Adversarial inputs exhaust model or serving resources, causing denial
of service fo
r legitimate users.",
        "Context flooding and pathological prompts degrade latency or
availability of an LLM
 endpoint.",
    ],
    "unbounded consumption": [
        "Attackers force unbounded token usage or recursive tool calls that
amplify cloud co
st and quota burn.",
        "Missing rate limits allow runaway consumption of LLM inference
budget.",
    ],
    "model backdoor": [
        "A trojaned or backdoored model activates malicious behavior when a
hidden trigger p
hrase appears.",
        "Supply-chain or fine-tuning backdoors implant sleeper behaviors into
deployed LLMs.
",
    ],
    "agent loop and cascading failure": [
        "Agents enter infinite planning or tool loops that cascade into
system-wide failure.
",
        "Recursive multi-step agent workflows amplify errors until the session
collapses.",
    ],
    "multi-agent communication attack": [
        "Malicious messages between agents inject instructions or corrupt
shared plans in a
multi-agent system.",
        "Inter-agent channels are abused to spread compromised goals across a
swarm.",
    ],
```

```
   "unsafe code generation and execution": [
      "An LLM generates and executes unsafe code that compromises the host
or exfiltrates
data.",
      "Automatic execution of model-written scripts enables remote code
execution via the
agent.",
   ],
   "sandbox escape": [
      "Code or tools invoked by an LLM agent escape the intended sandbox or
container isol
ation.",
```

```
      "Sandbox breakout turns a constrained agent runtime into host-level
compromise.",
   ],
   "plugin and mcp trust attack": [
      "A malicious MCP server or plugin abuses trust to steal context,
escalate tools, or
alter agent behavior.",
      "Compromised tool servers in the Model Context Protocol path become a
confused-deput
y channel.",
   ],
   "authentication and authorization bypass": [
      "LLM-mediated workflows bypass authentication or authorization checks
through prompt
 or tool abuse.",
      "Broken access control in agent apps lets users reach privileged
actions via crafted
 requests.",
   ],
   "unsafe file handling": [
      "Agents read or write arbitrary files through path traversal or unsafe
upload handli
ng.",
      "LLM tool file APIs expose sensitive host paths without adequate
validation.",
   ],
   "training and fine-tuning data poisoning": [
      "Poisoned fine-tuning or pretraining data implants targeted behaviors
into the resul
ting model.",
      "Adversaries contaminate datasets used to train or align large
language models.",
   ],
   "adversarial examples": [
      "Small input perturbations cause large language or multimodal models
to misclassify
or misbehave.",
      "Adversarial examples craft near-imperceptible changes that defeat
model robustness.
",
   ],
   "model extraction": [
      "Attackers query an API repeatedly to steal or approximate a
proprietary model's fun
ctionality.",
      "Model extraction reconstructs decision boundaries from black-box
access.",
   ],
   "membership inference": [
      "Membership inference determines whether a specific record was in the
model's traini
ng set.",
      "Privacy attacks infer training-set membership from model confidence
patterns.",
   ],
```

```
   "model inversion": [
      "Model inversion reconstructs sensitive training attributes or
examples from model o
utputs.",
      "Inversion attacks recover private information encoded in model
parameters or respon
ses.",
   ],
   "training data extraction": [
      "Training data extraction recovers verbatim memorized text from a
deployed language
model.",
      "Memorization attacks elicit confidential documents that were present
in training da
```

```
ta.",
    ],
    "system prompt leakage": [
        "Users coerce the model into revealing hidden system prompts or
developer instructio
ns.",
        "System prompt leakage discloses proprietary policies and tool
configurations.",
    ],
    "context isolation failure": [
        "Failures in context isolation leak data across users, tenants, or
sessions in an LL
M app.",
        "Shared caches or windows mix confidential contexts between unrelated
conversations.
",
    ],
    "memory poisoning": [
        "Attackers poison an agent's long-term memory so future sessions
follow malicious in
structions.",
        "Persistent memory stores become a durable channel for prompt
injection.",
    ],
    "agent impersonation": [
        "An attacker spoofs agent identity to issue trusted commands inside a
multi-agent sy
stem.",
        "Agent impersonation abuses weak identity binding between
collaborating agents.",
    ],
    "server-side request forgery": [
        "LLM tools that fetch URLs are abused for SSRF against internal
services.",
        "Server-side request forgery via agent browsing reaches cloud metadata
or intranet h
osts.",
    ],
    "internal network reconnaissance": [
        "Compromised agents scan internal networks and map services beyond the
intended peri
meter.",
        "Tool-enabled LLMs perform reconnaissance that reveals private
infrastructure.",
    ],
    "misinformation and citation fabrication": [
        "Models fabricate citations and spread misinformation that looks
authoritative.",
        "Citation fabrication invents papers and URLs that do not exist.",
    ],
    "insufficient logging and monitoring": [
        "Insufficient logging leaves LLM abuse, prompt injection, and tool
misuse undetectab
le.",
        "Missing audit trails prevent incident response for agentic systems.",
```

```
    ],
    "model routing and downgrade attack": [
        "Attackers manipulate routers to force weaker or unaligned models for
sensitive task
s.",
        "Model downgrade attacks bypass stronger safety tiers by steering
traffic to cheaper
 models.",
    ],
}
STOPWORDS = {
```

```
   "a", "about", "ai", "and", "for", "in", "large", "language", "llm",
"llms",
   "model", "models", "of", "on", "or", "security", "the", "to", "with",
}
def detect_topics(text: str) -> list[str]:
   normalized = text.lower()
   return [
      topic
      for topic, keywords in TOPIC_KEYWORDS.items()
      if any(keyword in normalized for keyword in keywords)
   ]
def tokenize(text: str) -> set[str]:
   return {
      token
      for token in re.findall(r"[a-zA-Z][a-zA-Z0-9_-]{2,}", text.lower())
      if token not in STOPWORDS
   }
def build_gap_query(topics: list[str]) -> str:
   topic_text = " OR ".join(topics)
   return f"{topic_text} LLM vulnerability evidence advisory paper"
def is_core_topic(topic: str) -> bool:
   return topic in CORE_SECURITY_TOPICS
#   backend/intel_agent/__init__.py
"""intel_agent: self-contained LLM-security intelligence collection agent.
A Claude Agent SDK-native, loop-engineered, self-evolving intelligence
collector.
This package has ZERO dependency on the legacy ``sufe_saads_crewai``
package: it
carries its own schemas, source clients, relevance filter, persistence, and
the
agentic loop / playbook memory that make it self-contained and independently
runnable.
"""
from __future__ import annotations
__all__ = ["__version__"]
__version__ = "0.1.0"
#   backend/intel_agent/agent/__init__.py
"""The agentic collection loop: per-round SDK sessions and the termination
critic."""
from __future__ import annotations
__all__ = ["IntelAgentLoop", "RoundSessionResult", "decide_termination"]
def __getattr__(name: str):
   if name in {"IntelAgentLoop", "RoundSessionResult"}:
      from .loop import IntelAgentLoop, RoundSessionResult
      return {"IntelAgentLoop": IntelAgentLoop, "RoundSessionResult":
RoundSessionResult}[
name]
   if name == "decide_termination":
      from .critic import decide_termination
      return decide_termination
   raise AttributeError(name)
#   backend/intel_agent/agent/critic.py
"""Round-termination critic: agent verdict tempered by deterministic safety
valves.
The critic (a fast, tool-forced decision) proposes continue/stop from
```

marginal
yield features. Its verdict is then overridden, in priority order, by hard valves: max_rounds > stall > coverage gate (topics below quota) > min_rounds.

```
If the SDK decision fails, a pure-rules verdict is used -- the fallback
contract.
"""
from __future__ import annotations
import json
from typing import Any
from ..runtime.structured import StructuredDecisionEngine
from ..schemas import SearchCompletenessAssessment, TerminationDecision
SYSTEM_PROMPT = (
    "You judge whether an LLM-security collection loop should run another
round. "
    "Continue only while the marginal new-relevant-items-per-call justifies
the API "
    "cost. Answer via the submit_decision tool."
)
def decide_termination(
    engine: StructuredDecisionEngine | None,
    digest: str,
    features: dict[str, Any],
    round_index: int,
    max_rounds: int,
    min_rounds: int,
    open_gaps: list[str],
    stalled: bool,
    api_budget_remains: bool,
    on_fallback: Any | None = None,
) -> SearchCompletenessAssessment:
    assessment = _agent_or_rules(
        engine, digest, features, round_index, max_rounds, open_gaps, stalled,
on_fallback
    )
    _apply_safety_valves(
        assessment, round_index, max_rounds, min_rounds, open_gaps, stalled,
api_budget_rema
ins
    )
    return assessment
def _agent_or_rules(
    engine: StructuredDecisionEngine | None,
    digest: str,
    features: dict[str, Any],
    round_index: int,
    max_rounds: int,
    open_gaps: list[str],
    stalled: bool,
    on_fallback: Any | None,
) -> SearchCompletenessAssessment:
    if engine is not None:
        prompt = (
            f"{digest}\n\nMarginal yield features: {json.dumps(features)}\n"
            f"Round {round_index + 1} of max {max_rounds}.\n\n"
            "Decide whether to continue for another round."
        )
        parsed = engine.decide(
            "termination", prompt, TerminationDecision,
```

```
system_prompt=SYSTEM_PROMPT, fast=Tr
```

```
ue
        )
        if parsed is not None:
            return SearchCompletenessAssessment(
                completeness_score=parsed.completeness_score,
                should_continue=parsed.should_continue,
                missing_dimensions=parsed.missing_topics,
                recommended_next_mode="gap_fill" if parsed.should_continue else
None,
                stop_rationale=parsed.stop_reason or parsed.rationale,
            )
        if on_fallback is not None:
            on_fallback("termination", "decision failed")
    # Deterministic fallback: continue while gaps remain and search is not
stalled.
    should_continue = bool(open_gaps) and not stalled
    return SearchCompletenessAssessment(
        completeness_score=0.0 if open_gaps else 1.0,
        should_continue=should_continue,
        missing_dimensions=open_gaps,
        recommended_next_mode="gap_fill" if should_continue else None,
        stop_rationale=None if should_continue else "no open coverage gaps or
search stalled
",
    )
def _apply_safety_valves(
    assessment: SearchCompletenessAssessment,
    round_index: int,
    max_rounds: int,
    min_rounds: int,
    open_gaps: list[str],
    stalled: bool,
    api_budget_remains: bool,
) -> None:
    if round_index + 1 >= max_rounds:
        assessment.should_continue = False
        assessment.stop_rationale = assessment.stop_rationale or "max_rounds
reached"
        return
    if stalled:
        assessment.should_continue = False
        reason = "search exhausted: marginal new-relevant yield below
threshold"
        if open_gaps:
            reason += f"; unreached topics: {open_gaps}"
        assessment.stop_rationale = reason
        assessment.missing_dimensions = open_gaps
        return
    if open_gaps and api_budget_remains:
        assessment.should_continue = True
        assessment.stop_rationale = None
        assessment.missing_dimensions = open_gaps
        return
    if round_index + 1 < min_rounds:
        assessment.should_continue = True
```

```
#   backend/intel_agent/agent/loop.py
"""One agentic collection round = one claude-agent-sdk session.
The agent calls the source/memory/control tools itself inside the SDK's
native
tool loop; results accumulate in the shared ``ToolContext``. A fresh session
per
round keeps cost bounded and every decision input (the digest) reproducible.
Hard session failures with no collected items degrade to the deterministic
rules
path; turn-limit / truncated sessions that already gathered items soft-
complete
so the controller keeps the agent yield instead of re-running rules
fallback.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
import asyncio
import concurrent.futures
import threading
from dataclasses import dataclass
from typing import Any
from ..observability import TraceSink
from ..runtime import client as runtime_client
from ..runtime.structured import _run_coro_sync
from ..hooks import build_hooks
from ..tools.context import ToolContext
from ..tools.server import build_round_server
from .system_prompt import SYSTEM_PROMPT
DEFAULT_ROUND_TIMEOUT_SECONDS = 300.0
# Search-heavy bootstrap rounds often need many parallel tool batches; 12
was
# too tight and triggered "Reached maximum number of turns" mid-sweep.
DEFAULT_MAX_TURNS = 24
def resolve_max_turns(explicit: int | None = None) -> int:
    if explicit is not None:
        return max(1, int(explicit))
    raw = (_setting("INTEL_AGENT_MAX_TURNS") or "").strip()
    if raw.isdigit():
        return max(1, int(raw))
    return DEFAULT_MAX_TURNS
@dataclass
class RoundSessionResult:
    completed: bool
    assistant_text: str
    error: str | None = None
class IntelAgentLoop:
    def __init__(
        self,
        model: str | None = None,
        max_turns: int | None = None,
        timeout_seconds: float = DEFAULT_ROUND_TIMEOUT_SECONDS,
        runner: Any | None = None,
        client_factory: Any | None = None,
    ) -> None:
        self.model = model or runtime_client.main_model()
```

```
        self.max_turns = resolve_max_turns(max_turns)
```

```
        self.timeout_seconds = timeout_seconds
        # Test seam: async (ctx, prompt, model, max_turns) -> str (assistant
text)
        self._runner = runner
        self._client_factory = client_factory
        self._session: _ContinuousSDKSession | None = None
        self._continuous_enabled = True
    def available(self) -> bool:
        if self._runner is not None or self._client_factory is not None:
            return True
        ok, _ = runtime_client.sdk_runtime_available()
        return ok
    def start_run(self, run_id: str, continuous: bool = True) -> None:
        if self._runner is not None:
            return
        self.end_run()
        self._continuous_enabled = continuous
        if not continuous:
            return
        self._session = _ContinuousSDKSession(
            model=self.model,
            max_turns=self.max_turns,
            timeout_seconds=self.timeout_seconds,
            client_factory=self._client_factory,
        )
    def end_run(self) -> None:
        if self._session is not None:
            self._session.close()
            self._session = None
    @property
    def session_id(self) -> str | None:
        return self._session.session_id if self._session is not None else None
    def run_round(
        self, ctx: ToolContext, prompt: str, trace: TraceSink | None = None
    ) -> RoundSessionResult:
        if not self.available():
            return RoundSessionResult(False, "", "sdk runtime unavailable")
        try:
            if self._runner is None:
                if self._continuous_enabled:
                    if self._session is None:
                        self.start_run(ctx.run_id)
                    assert self._session is not None
                    text = self._session.run_round(ctx, prompt, trace)
                else:
                    text = _run_coro_sync(
                        _one_off_session_runner(
                            ctx, prompt, self.model, self.max_turns, trace
                        ),
                        timeout_seconds=self.timeout_seconds,
                    )
```

```
          else:
              text = _run_coro_sync(
                  self._runner(ctx, prompt, self.model, self.max_turns),
                  timeout_seconds=self.timeout_seconds,
              )
       except Exception as exc:  # noqa: BLE001 - round-level fallback /
soft-complete
          return _finalize_round(ctx, "", f"{exc.__class__.__name__}:
{exc}", trace)
       return _finalize_round(ctx, text or "", None, trace)
def _finalize_round(
   ctx: ToolContext,
   text: str,
   error: str | None,
   trace: TraceSink | None,
) -> RoundSessionResult:
   """Keep agent yield on turn-limit / SDK error if searches already wrote
items."""
   has_yield = bool(ctx.collected_items) or bool(ctx.notes)
   if error and has_yield:
      _ensure_truncated_note(ctx, error)
      if trace is not None:
         trace.emit(
            "session_truncated",
            reason=error,
            collected=len(ctx.collected_items),
         )
      return RoundSessionResult(True, text, error)
   if error:
      return RoundSessionResult(False, text, error)
   return RoundSessionResult(has_yield, text, None)
def _ensure_truncated_note(ctx: ToolContext, error: str) -> None:
   if ctx.notes:
      return
   ctx.notes.append(
      {
         "summary": (
            f"Round truncated ({error[:160]}). Kept
{len(ctx.collected_items)} "
            "items already collected before the session ended."
         ),
         "covered_topics": [],
         "remaining_gaps": list(ctx.target_topics),
      }
   )
def _short_tool_name(name: str) -> str:
   return name.rsplit("__", 1)[-1] if name.startswith("mcp__") else name
async def _one_off_session_runner(
   ctx: ToolContext,
   prompt: str,
   model: str,
   max_turns: int,
   trace: TraceSink | None,
) -> str:
```

```
   """Legacy/full behavior: one isolated SDK query per controller round."""
   from claude_agent_sdk import (
      AssistantMessage,
      ResultMessage,
      TextBlock,
      ThinkingBlock,
      ToolResultBlock,
      ToolUseBlock,
      UserMessage,
      query,
   )
   mcp_servers, allowed_tools = build_round_server(ctx)
   options = runtime_client.build_options(
      model=model,
      max_turns=max_turns,
      system_prompt=SYSTEM_PROMPT,
      mcp_servers=mcp_servers,
      allowed_tools=allowed_tools,
      hooks=build_hooks(ctx),
   )
   texts: list[str] = []
   async for message in query(prompt=prompt, options=options):
      if isinstance(message, AssistantMessage):
         for block in message.content:
            if isinstance(block, TextBlock):
               texts.append(block.text)
               if trace and block.text.strip():
                  trace.emit("assistant_text", text=block.text)
            elif isinstance(block, ThinkingBlock) and trace:
               trace.emit("thinking", text=block.thinking)
            elif isinstance(block, ToolUseBlock) and trace:
               trace.emit("tool_call", tool=_short_tool_name(block.name),
input=block.i
nput)
      elif isinstance(message, UserMessage) and trace and
isinstance(message.content, list
):
         for block in message.content:
            if isinstance(block, ToolResultBlock):
               trace.emit("tool_result", content=block.content,
is_error=block.is_error
)
      elif isinstance(message, ResultMessage):
         if trace:
            trace.emit(
               "session_end",
               turns=message.num_turns,
               cost_usd=message.total_cost_usd,
               is_error=message.is_error,
            )
         if message.is_error and not ctx.collected_items and not ctx.notes:
            raise RuntimeError(f"sdk session error: {str(message.result)
[:200]}")
   return "\n".join(texts)
```

```
class _ToolContextProxy:
   """Stable object captured by MCP handlers while the active round context
changes."""
   def __init__(self) -> None:
      object.__setattr__(self, "_current", None)
   def bind(self, ctx: ToolContext) -> None:
      object.__setattr__(self, "_current", ctx)
   def __getattr__(self, name: str) -> Any:
      current = object.__getattribute__(self, "_current")
      if current is None:
         raise RuntimeError("SDK tool context is not bound to a round")
      return getattr(current, name)
   def __setattr__(self, name: str, value: Any) -> None:
      if name == "_current":
         object.__setattr__(self, name, value)
         return
      current = object.__getattribute__(self, "_current")
      if current is None:
         raise RuntimeError("SDK tool context is not bound to a round")
      setattr(current, name, value)
class _AsyncWorker:
   def __init__(self) -> None:
      self.loop = asyncio.new_event_loop()
      self.thread = threading.Thread(target=self._run, name="intel-agent-
sdk", daemon=True
)
      self.thread.start()
   def _run(self) -> None:
      asyncio.set_event_loop(self.loop)
      self.loop.run_forever()
   def submit(self, coro: Any, timeout: float) -> Any:
      future = asyncio.run_coroutine_threadsafe(coro, self.loop)
      try:
         return future.result(timeout=timeout)
      except concurrent.futures.TimeoutError:
         future.cancel()
         raise TimeoutError(f"SDK round exceeded {timeout:.0f}s") from None
   def close(self) -> None:
      if self.loop.is_running():
         self.loop.call_soon_threadsafe(self.loop.stop)
         self.thread.join(timeout=5)
      if not self.loop.is_closed():
         self.loop.close()
class _ContinuousSDKSession:
   """One ClaudeSDKClient and event loop for the lifetime of an incremental
run."""
   def __init__(
      self,
      model: str,
      max_turns: int,
      timeout_seconds: float,
      client_factory: Any | None = None,
   ) -> None:
```

```
        self.model = model
        self.max_turns = max_turns
        self.timeout_seconds = timeout_seconds
        self.proxy = _ToolContextProxy()
        self.worker = _AsyncWorker()
        self.client: Any | None = None
        self.session_id: str | None = None
        self.client_factory = client_factory
    def run_round(
        self, ctx: ToolContext, prompt: str, trace: TraceSink | None
    ) -> str:
        self.proxy.bind(ctx)
        return str(self.worker.submit(self._run_with_recovery(prompt, trace),
self.timeout_s
econds))
    async def _connect(self, resume: str | None = None) -> None:
        from claude_agent_sdk import ClaudeSDKClient
        mcp_servers, allowed_tools = build_round_server(self.proxy)  # type:
ignore[arg-type
]
        options = runtime_client.build_options(
            model=self.model,
            max_turns=self.max_turns,
            system_prompt=SYSTEM_PROMPT,
            mcp_servers=mcp_servers,
            allowed_tools=allowed_tools,
            hooks=build_hooks(self.proxy),  # type: ignore[arg-type]
            resume=resume,
        )
        factory = self.client_factory or ClaudeSDKClient
        self.client = factory(options=options)
        await self.client.connect()
    async def _disconnect(self) -> None:
        if self.client is not None:
            try:
                await self.client.disconnect()
            finally:
                self.client = None
    async def _run_with_recovery(self, prompt: str, trace: TraceSink | None)
-> str:
        if self.client is None:
            await self._connect()
        try:
            return await self._query_once(prompt, trace)
        except Exception as original:
            if self.proxy.collected_items or self.proxy.notes:
                raise
            last_error: Exception = original
            recovery_modes = [self.session_id] if self.session_id else []
            recovery_modes.append(None)
            for resume in recovery_modes:
                try:
                    await self._disconnect()
```

```
                await self._connect(resume=resume)
                if trace is not None:
                    trace.emit("session_recovery", mode="resume" if resume
else "fresh")
                return await self._query_once(prompt, trace)
            except Exception as exc:  # noqa: BLE001 - try next recovery
tier
                last_error = exc
        raise last_error from original
   async def _query_once(self, prompt: str, trace: TraceSink | None) -> str:
       from claude_agent_sdk import (
           AssistantMessage,
           ResultMessage,
           TextBlock,
           ThinkingBlock,
           ToolResultBlock,
           ToolUseBlock,
           UserMessage,
       )
       assert self.client is not None
       await self.client.query(prompt)
       texts: list[str] = []
       async for message in self.client.receive_response():
           if isinstance(message, AssistantMessage):
               for block in message.content:
                   if isinstance(block, TextBlock):
                       texts.append(block.text)
                       if trace and block.text.strip():
                           trace.emit("assistant_text", text=block.text)
                   elif isinstance(block, ThinkingBlock) and trace:
                       trace.emit("thinking", text=block.thinking)
                   elif isinstance(block, ToolUseBlock) and trace:
                       trace.emit("tool_call",
tool=_short_tool_name(block.name), input=blo
ck.input)
           elif isinstance(message, UserMessage) and trace and
isinstance(message.content,
list):
               for block in message.content:
                   if isinstance(block, ToolResultBlock):
                       trace.emit("tool_result", content=block.content,
is_error=block.is_e
rror)
           elif isinstance(message, ResultMessage):
               self.session_id = message.session_id or self.session_id
               if trace:
                   trace.emit(
                       "session_end",
                       turns=message.num_turns,
                       cost_usd=message.total_cost_usd,
                       is_error=message.is_error,
                   )
           # Empty yield + error → hard fail (controller rules fallback).
           # Non-empty yield → return normally; if the SDK also raises after
this
```

```
# message, run_round soft-completes via _finalize_round.
```

```
            if message.is_error and not self.proxy.collected_items and not
self.proxy.no
tes:
                raise RuntimeError(f"sdk session error: {str(message.result)
[:200]}")
      return "\n".join(texts)
   def close(self) -> None:
      try:
         if self.client is not None:
            self.worker.submit(self._disconnect(), timeout=10)
      finally:
         self.worker.close()
#   backend/intel_agent/agent/system_prompt.py
"""Fixed system prompt (cache-friendly) plus per-round brief rendering."""
from __future__ import annotations
SYSTEM_PROMPT = (
   "You are the collection brain of an LLM-security intelligence agent. Each
turn "
   "you run ONE round of collection by calling the provided search tools
directly, "
   "observing results, and adapting your next query -- a tight search loop.
\n\n"
   "Tools:\n"
   "- get_collection_state / score_query_candidates (adaptive runs): inspect
the "
   "authoritative gaps, then submit 1-8 candidates for deterministic UCB
approval. "
   "Only call source tools with an approved candidate_id. Put every source-
specific "
   "operator inside the candidate's params object; never add operator fields
at the "
   "candidate top level.\n"
   "- search_nvd / search_arxiv / search_cisa_kev / search_osv: query
registered "
   "sources with typed advanced operators. Combine operators (CWE, severity,
date "
   "windows, arXiv field/category filters, OSV ecosystem/package) to raise
recall "
   "on relevant LLM-security intel while keeping noise low.\n"
   "- recall_playbook: retrieve techniques that worked in past runs BEFORE
planning.\n"
   "- record_technique: save a technique that just worked, grounded in
observed results.\n"
   "- submit_round_summary: call EXACTLY ONCE at the end to finish the
round.\n\n"
   "Principles:\n"
   "1. Start by recalling the playbook. In adaptive runs inspect collection
state, "
   "submit source-specific candidates for scoring, then execute only
approvals.\n"
   "2. Maximize NEW RELEVANT items per API call. Do not repeat earlier
queries.\n"
   "3. Prefer operator combinations the digest/playbook show worked (high
new/call, "
   "low noise). Spread across sources rather than hammering one.\n"
```

```
    "4. Objective: capture as much LLM/AI-security intelligence as possible
while "
    "admitting as little unrelated content as possible.\n"
    "5. Ground every decision in the digest you are given; never fabricate
CVE IDs, "
    "results, or identifiers.\n"
    "6. Budget your tool turns carefully. Batch parallel searches, stop when
"
    "marginal yield drops, and ALWAYS call submit_round_summary before you
run out "
    "of turns -- never leave a round without that closing call."
)
FULL_MODE_BRIEF = (
    "MODE: full collection. Coverage quotas apply to Core LLM-security topics
"
    "(prompt injection, jailbreak, agent tool abuse, data leakage, model
supply "
    "chain, rag poisoning, insecure output handling, excessive agency, model
DoS, "
    "unbounded consumption, model backdoor, agent loop/cascading failure, "
    "multi-agent communication attack, unsafe code gen/exec, sandbox escape,
"
```

```
    "plugin/MCP trust, authz bypass, unsafe file handling). Also search and
label "
    "Extended threats (SSRF, model extraction, memory poisoning, etc.) when
they "
    "appear, but prioritize open Core coverage gaps. Maximize relevant recall
while "
    "suppressing unrelated noise."
)
INCREMENTAL_MODE_BRIEF = (
    "MODE: incremental collection. Only collect items within the given time
scope "
    "and focus. Core topics drive CorpusGap priority; Extended topics are
searchable "
    "and labelable but NEVER require coverage. Use native date operators and
source "
    "checkpoints; optimize verified gap reduction rather than raw result
count."
)
def render_round_prompt(
    digest: str,
    mode_brief: str,
    round_index: int,
    max_rounds: int,
    focus: str | None = None,
    time_scope_hint: str | None = None,
    open_gaps: list[str] | None = None,
    max_turns: int | None = None,
) -> str:
    lines = [mode_brief, ""]
    if focus:
        lines.append(f"Focus entity/topic: {focus}")
    if time_scope_hint:
        lines.append(f"Time scope: {time_scope_hint}")
    if open_gaps:
        lines.append(f"Target topics still below quota (prioritize these):
{open_gaps}")
    lines.append("")
    lines.append(digest)
    lines.append("")
    turn_budget = (
        f" You have at most {max_turns} tool turns this session;"
        if max_turns
        else ""
    )
    lines.append(
        f"This is round {round_index + 1} of at most {max_rounds}.
{turn_budget} "
        "Recall the playbook, run a focused set of searches (batch in
parallel), "
        "record any technique that worked, then call submit_round_summary to
end "
        "the round before the turn budget is exhausted."
    )
    return "\n".join(lines)
#   backend/intel_agent/analysis.py
```

```
"""Deterministic analysis + fallback helpers for the collection loop.
A lean, self-contained replacement for the legacy ``intel/rules.py``: it
keeps
only the pure functions the agentic controller needs -- batch merging with
novelty/noise/duplicate accounting, quota-based coverage gaps, stall
detection,
and deterministic query building for the rules-only fallback round.
"""
```

```
from __future__ import annotations
from intel_agent.runtime.client import _setting
import json
from dataclasses import dataclass, field
from typing import Any
from .schemas import (
    CoverageGap,
    IntelRunBlackboard,
    QueryHistoryEntry,
    RawIntelItem,
    RawIntelItemBatch,
    SearchQueryPlan,
)
from .topics import ALL_SECURITY_TOPICS, CORE_SECURITY_TOPICS,
build_gap_query, detect_topic
s
RELEVANT_SCORE_THRESHOLD = 0.5
DEFAULT_COVERAGE_QUOTA = 3
@dataclass(frozen=True)
class SourceQuerySpec:
    source_name: str
    query_text: str
    target_topics: list[str]
    max_results: int
    strategy_name: str = "rules"
    params: dict[str, Any] = field(default_factory=dict)
# ------------------------------------------------------------------ env
knobs
def _env_int(name: str, default: int) -> int:
    raw = _setting(name)
    if not raw:
        return default
    try:
        return max(0, int(raw))
    except ValueError:
        return default
def _env_float(name: str, default: float) -> float:
    raw = _setting(name)
    if not raw:
        return default
    try:
        return float(raw)
    except ValueError:
        return default
def coverage_quota() -> int:
    return _env_int("INTEL_COVERAGE_QUOTA", DEFAULT_COVERAGE_QUOTA)
def stall_patience() -> int:
    return _env_int("INTEL_STALL_PATIENCE", 2)
def min_new_relevant_per_call() -> float:
    return _env_float("INTEL_MIN_NEW_RELEVANT_PER_CALL", 0.5)
# ----------------------------------------------------------------------
relevance
def item_is_relevant(item: RawIntelItem) -> bool:
```

```
    relevance = item.metadata.get("relevance")
    if isinstance(relevance, dict) and relevance.get("label"):
        return relevance["label"] == "relevant"
    return float(item.relevance_score) >= RELEVANT_SCORE_THRESHOLD
def item_topics(item: RawIntelItem, target_topics: list[str]) -> set[str]:
    topics: set[str] = set()
    relevance = item.metadata.get("relevance")
    if isinstance(relevance, dict) and relevance.get("topic") in
target_topics:
        topics.add(relevance["topic"])
    topics.update(t for t in item.metadata.get("topics", []) if t in
target_topics)
    topics.update(detect_topics(f"{item.title} {item.summary} {item.raw_text
or ''}"))
    return topics & set(target_topics)
def operator_signature(source_name: str, params: dict[str, Any]) -> str:
    keys = sorted(key for key, value in params.items() if value not in (None,
"", [], {}))
    return f"{source_name}[{'+'.join(keys)}]" if keys else f"{source_name}
[base]"
def source_query_key(spec: SourceQuerySpec) -> str:
    return json.dumps(
        {"source": spec.source_name, "query": spec.query_text, "params":
spec.params},
        ensure_ascii=False, sort_keys=True,
    )
# ------------------------------------------------------------------ merging
def merge_batch_into_blackboard(
    blackboard: IntelRunBlackboard,
    query_plan: SearchQueryPlan,
    batch: RawIntelItemBatch,
) -> QueryHistoryEntry:
    """Add newly-seen items and record a query-history entry with round
metrics."""
    existing_ids = {item.item_id for item in blackboard.raw_items}
    new_items = [item for item in batch.items if item.item_id not in
existing_ids]
    blackboard.raw_items.extend(new_items)
    total = len(batch.items)
    new_count = len(new_items)
    relevant_new = sum(1 for item in new_items if item_is_relevant(item))
    novelty = new_count / total if total else 0.0
    duplicate_ratio = (total - new_count) / total if total else 0.0
    noise_ratio = (new_count - relevant_new) / new_count if new_count else
0.0
    source_plans = [
        {"source_name": stat.source_name, "params": {}} for stat in
batch.source_stats
    ] or [{"source_name": name, "params": {}} for name in
query_plan.source_names]
    entry = QueryHistoryEntry(
        query_text=query_plan.query_text,
        source_names=query_plan.source_names,
        result_count=total,
        novelty_score=round(novelty, 4),
```

```
        noise_ratio=round(noise_ratio, 4),
        duplicate_ratio=round(duplicate_ratio, 4),
        round_index=query_plan.round_index,
        metadata={
            "new_item_ids": [item.item_id for item in new_items],
            "relevant_new": relevant_new,
```

```
            "target_topics": query_plan.target_topics,
            "source_query_plans": source_plans,
        },
    )
    blackboard.query_history.append(entry)
    return entry
def combine_batches(batches: list[RawIntelItemBatch]) -> RawIntelItemBatch:
    items: list[RawIntelItem] = []
    stats = []
    seen: set[str] = set()
    for batch in batches:
        for item in batch.items:
            if item.item_id in seen:
                continue
            seen.add(item.item_id)
            items.append(item)
        stats.extend(batch.source_stats)
    return RawIntelItemBatch(items=items, source_stats=stats)
# ---------------------------------------------------------------------
coverage
def topic_coverage_counts(
    items: list[RawIntelItem], target_topics: list[str]
) -> dict[str, int]:
    counts = {topic: 0 for topic in target_topics}
    for item in items:
        if not item_is_relevant(item):
            continue
        for topic in item_topics(item, target_topics):
            counts[topic] += 1
    return counts
def quota_open_gaps(
    items: list[RawIntelItem], target_topics: list[str], quota: int
) -> list[str]:
    counts = topic_coverage_counts(items, target_topics)
    return [topic for topic in target_topics if counts.get(topic, 0) < quota]
def analyze_coverage_gaps(
    items: list[RawIntelItem], target_topics: list[str], quota: int
) -> list[CoverageGap]:
    counts = topic_coverage_counts(items, target_topics)
    gaps: list[CoverageGap] = []
    for topic in target_topics:
        count = counts.get(topic, 0)
        if count >= quota:
            continue
        coverage = count / quota if quota else 0.0
        gaps.append(
            CoverageGap(
                gap_id=f"gap:{topic.replace(' ', '_')}",
                taxonomy_or_component=topic,
                current_coverage=round(coverage, 3),
                estimated_gap_fill_roi=round(1.0 - coverage, 3),
```

```
            priority="high" if count == 0 else "medium",
         )
      )
   return gaps
# ---------------------------------------------------------------- stall
def is_search_stalled(
   blackboard: IntelRunBlackboard, patience: int, min_new_relevant_per_call:
float
) -> bool:
   history = blackboard.query_history
   if len(history) < patience:
      return False
   for entry in history[-patience:]:
      calls = len(entry.metadata.get("source_query_plans") or []) or
len(entry.source_name
s) or 1
      relevant_new = int(entry.metadata.get("relevant_new", 0))
      if relevant_new / calls >= min_new_relevant_per_call:
         return False
   return True
def new_relevant_per_call(entry: QueryHistoryEntry) -> float:
   calls = len(entry.metadata.get("source_query_plans") or []) or
len(entry.source_names) o
r 1
   return int(entry.metadata.get("relevant_new", 0)) / calls
# ----------------------------------------------------------------
fallback
def build_fallback_specs(
   blackboard: IntelRunBlackboard,
   query_plan: SearchQueryPlan,
   target_topics: list[str],
   max_results: int,
   executed_keys: set[str],
) -> list[SourceQuerySpec]:
   """Deterministic breadth sweep used by the rules-only fallback round."""
   open_gaps = quota_open_gaps(blackboard.raw_items, target_topics,
coverage_quota())
   focus_topics = open_gaps or target_topics
   query_text = query_plan.query_text or build_gap_query(focus_topics[:3])
   per_source = max(5, min(20, max_results // 4))
   specs: list[SourceQuerySpec] = []
   for source_name in query_plan.source_names or list(_DEFAULT_SOURCES):
      spec = SourceQuerySpec(
         source_name=source_name,
         query_text=query_text,
         target_topics=focus_topics,
         max_results=per_source,
         strategy_name="rules_fallback",
         params={},
      )
      if source_query_key(spec) in executed_keys:
         continue
      specs.append(spec)
   return specs
_DEFAULT_SOURCES = ("nvd_cve_api", "arxiv_api", "cisa_kev_json",
```

```
"osv_dev_api")
```

```
def topic_bucket(topics: list[str]) -> str:
    lowered = " ".join(topics).lower()
    for topic in CORE_SECURITY_TOPICS:
        if topic in lowered:
            return topic
    for topic in ALL_SECURITY_TOPICS:
        if topic in lowered:
            return topic
    return "general"
#   backend/intel_agent/cli.py
"""Command-line entry points for the intel_agent collection engine.
    intel-agent full --max-rounds 6 --max-api-calls 40
    intel-agent incremental --focus "agent tool abuse" --window-days 1
    intel-agent latest
"""
from __future__ import annotations
import argparse
from datetime import datetime, timezone
from .engine.controller import IntelAgentController
from .engine.modes import FullCollectionMode, IncrementalCollectionMode
from .observability import TraceSink
from .runtime import client as runtime_client
from .schemas import RunBudget
from .store import default_store
def _parse_since(value: str | None) -> datetime | None:
    if not value:
        return None
    raw = value.replace("Z", "+00:00")
    parsed = datetime.fromisoformat(raw)
    return parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None else
parsed
def _make_trace(args: argparse.Namespace) -> TraceSink | None:
    return TraceSink(console=True) if getattr(args, "verbose", False) else
None
def _run_full(args: argparse.Namespace) -> int:
    controller = IntelAgentController()
    blackboard = controller.run(
        mode=FullCollectionMode(),
        budget=RunBudget(max_rounds=args.max_rounds,
max_api_calls=args.max_api_calls),
        run_id=args.run_id,
        trace=_make_trace(args),
    )
    _report(blackboard)
    return 0
def _run_incremental(args: argparse.Namespace) -> int:
    controller = IntelAgentController()
    mode = IncrementalCollectionMode(
        focus=args.focus,
        window_days=args.window_days,
        since=_parse_since(args.since),
        use_watermark=not args.no_watermark,
    )
```

```
    blackboard = controller.run(
        mode=mode,
        budget=RunBudget(max_rounds=args.max_rounds,
max_api_calls=args.max_api_calls),
        run_id=args.run_id,
        trace=_make_trace(args),
    )
    _report(blackboard)
    return 0
def _show_latest(_: argparse.Namespace) -> int:
    print(default_store().format_latest_intel(limit=20))
    return 0
def _report(blackboard) -> None:
    print(f"\nRun {blackboard.run_id} [{blackboard.run_mode}] finished.")
    print(f"  rounds: {len(blackboard.query_history)}")
    print(f"  items collected: {len(blackboard.raw_items)}")
    print(f"  api calls used: {blackboard.metrics.api_calls_used}")
    if blackboard.coverage_gaps:
        print(f"  open coverage gaps: {[g.taxonomy_or_component for g in
blackboard.coverage
_gaps]}")
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="intel-agent", description=__doc__)
    subparsers = parser.add_subparsers(dest="command", required=True)
    def add_common(sub: argparse.ArgumentParser) -> None:
        sub.add_argument("--max-rounds", type=int, default=6)
        sub.add_argument("--max-api-calls", type=int, default=40)
        sub.add_argument("--run-id", type=str, default=None)
        sub.add_argument(
            "--verbose", "-v", action="store_true",
            help="Stream per-turn model text + tool calls; also dump "
            "data/intel_agent/traces/<run_id>.jsonl.",
        )
    full = subparsers.add_parser("full", help="Full LLM-security collection
across all topic
s.")
    add_common(full)
    full.set_defaults(func=_run_full)
    inc = subparsers.add_parser("incremental", help="Incremental collection
by focus + time
window.")
    add_common(inc)
    inc.add_argument("--focus", type=str, default=None, help="Topic/entity
focus, e.g. 'jail
break'.")
    inc.add_argument("--window-days", type=int, default=1, help="Look-back
window in days.")
    inc.add_argument("--since", type=str, default=None, help="Explicit lower
bound (ISO-8601
).")
    inc.add_argument("--no-watermark", action="store_true", help="Ignore
prior-run watermark
.")
    inc.set_defaults(func=_run_incremental)
    latest = subparsers.add_parser("latest", help="Show the most recent
```

```
persisted run.")
    latest.set_defaults(func=_show_latest)
    return parser
def main(argv: list[str] | None = None) -> int:
```

```
    args = build_parser().parse_args(argv)
    if getattr(args, "command", None) in {"full", "incremental"}:
        ok, reason = runtime_client.sdk_runtime_available()
        if not ok:
            print(f"[warn] SDK runtime unavailable ({reason}); running
deterministic fallbac
k.")
    return args.func(args)
def run_full() -> None:
    raise SystemExit(main(["full"]))
def run_incremental() -> None:
    raise SystemExit(main(["incremental"]))
def show_latest() -> None:
    raise SystemExit(main(["latest"]))
if __name__ == "__main__":
    raise SystemExit(main())
#   backend/intel_agent/engine/__init__.py
"""Engine layer: modes, round orchestration, context digest, rules
fallback."""
from __future__ import annotations
__all__ = [
    "IntelAgentController",
    "CollectionMode",
    "FullCollectionMode",
    "IncrementalCollectionMode",
]
def __getattr__(name: str):
    if name == "IntelAgentController":
        from .controller import IntelAgentController
        return IntelAgentController
    if name in {"CollectionMode", "FullCollectionMode",
"IncrementalCollectionMode"}:
        from .modes import CollectionMode, FullCollectionMode,
IncrementalCollectionMode
        return {
            "CollectionMode": CollectionMode,
            "FullCollectionMode": FullCollectionMode,
            "IncrementalCollectionMode": IncrementalCollectionMode,
        }[name]
    raise AttributeError(name)
#   backend/intel_agent/engine/bandit.py
"""Deterministic UCB ranking and reward attribution for adaptive queries."""
from __future__ import annotations
import math
from collections import defaultdict
from ..analysis import operator_signature
from ..schemas import CorpusGap, QueryCallOutcome, QueryCandidate
def arm_key(candidate: QueryCandidate) -> str:
    topic = candidate.target_topics[0] if candidate.target_topics else
"general"
    return "|".join(
        [
            topic,
            candidate.evidence_channel,
            candidate.source_name,
```

```
            candidate.query_intent,
            operator_signature(candidate.source_name, candidate.params),
        ]
    )
def _outcome_arm(outcome: QueryCallOutcome) -> str:
    topic = outcome.target_topics[0] if outcome.target_topics else "general"
    return "|".join(
        [topic, outcome.evidence_channel, outcome.source_name,
outcome.query_intent, outcome
.operator_signature]
    )
def score_candidates(
    candidates: list[QueryCandidate],
    outcomes: list[QueryCallOutcome],
    corpus_gaps: list[CorpusGap],
) -> list[tuple[QueryCandidate, float]]:
    stats: dict[str, list[float]] = defaultdict(list)
    for outcome in outcomes:
        stats[_outcome_arm(outcome)].append(float(outcome.reward))
    total_calls = len(outcomes)
    gaps = {gap.gap_id: gap for gap in corpus_gaps}
    ranked: list[tuple[QueryCandidate, float]] = []
    for candidate in candidates:
        rewards = stats.get(arm_key(candidate), [])
        history_mean = sum(rewards) / len(rewards) if rewards else 0.0
        target_priority = sum(
            gaps[gap_id].priority_score
            for gap_id in candidate.target_gap_ids
            if gap_id in gaps and gaps[gap_id].status == "open"
        )
        if not candidate.target_gap_ids and candidate.query_intent ==
"discovery":
            target_priority = 0.2
        expected_reduction = min(1.0, 0.6 * target_priority + 0.4 * max(0.0,
history_mean))
        exploration = math.sqrt(2.0 * math.log(total_calls + 2) /
(len(rewards) + 1))
        historical = [outcome for outcome in outcomes if _outcome_arm(outcome)
== arm_key(ca
ndidate)]
        noise = (
            sum(outcome.noise_count / max(1, outcome.new_count) for outcome in
historical) /
 len(historical)
            if historical else 0.0
        )
        duplicate = (
            sum(outcome.duplicate_count / max(1, outcome.in_scope_count) for
outcome in hist
orical)
            / len(historical)
            if historical else 0.0
        )
        utility = expected_reduction - 0.2 * noise - 0.2 * duplicate +
exploration
```

```
        ranked.append((candidate, round(utility, 4)))
    return sorted(ranked, key=lambda row: (-row[1], row[0].candidate_id))
def select_portfolio(
```

```
    candidates: list[QueryCandidate],
    outcomes: list[QueryCallOutcome],
    corpus_gaps: list[CorpusGap],
    max_calls: int = 4,
    max_total_calls: int | None = None,
    discovery_unbounded: bool = False,
    budget_outcomes: list[QueryCallOutcome] | None = None,
) -> list[tuple[QueryCandidate, float]]:
    ranked = score_candidates(candidates, outcomes, corpus_gaps)
    selected: list[tuple[QueryCandidate, float]] = []
    total_budget = max_total_calls or max(1, len(outcomes) + max_calls)
    prior_counts: dict[str, int] = defaultdict(int)
    for outcome in budget_outcomes if budget_outcomes is not None else
outcomes:
        prior_counts[outcome.query_intent] += 1
    discovery_remaining = max(0, math.ceil(total_budget * 0.20) -
prior_counts["discovery"])
    confirmation_remaining = max(0, math.ceil(total_budget * 0.10) -
prior_counts["confirmat
ion"])
    intent_caps = {
        "discovery": max_calls if discovery_unbounded else min(1,
discovery_remaining),
        "confirmation": min(1, confirmation_remaining),
    }
    intent_counts: dict[str, int] = defaultdict(int)
    for row in ranked:
        candidate = row[0]
        cap = intent_caps.get(candidate.query_intent)
        if cap is not None and intent_counts[candidate.query_intent] >= cap:
            continue
        selected.append(row)
        intent_counts[candidate.query_intent] += 1
        if len(selected) >= max_calls:
            break
    return selected
def normalized_reward(
    before_priority: float,
    after_priority: float,
    outcome: QueryCallOutcome,
    residual_support: float = 0.0,
) -> float:
    denominator = max(1.0, before_priority)
    gap_reduction = max(0.0, before_priority - after_priority) / denominator
    noise_ratio = outcome.noise_count / max(1, outcome.new_count)
    duplicate_ratio = outcome.duplicate_count / max(1,
outcome.in_scope_count)
    discovery_bonus = 0.0
    if outcome.query_intent == "discovery":
        extended_new = int(outcome.metadata.get("extended_new_count") or 0)
        discovery_bonus = min(0.25, 0.05 * extended_new + 0.1 *
residual_support)
    confirmation_bonus = (
        min(0.25, 0.1 * outcome.relevant_new)
        if outcome.query_intent == "confirmation" and outcome.success
```

```
        else 0.0
```

```
    in_scope_count: NonNegativeInt = 0
    duplicate_count: NonNegativeInt = 0
    new_count: NonNegativeInt = 0
    relevant_new: NonNegativeInt = 0
    noise_count: NonNegativeInt = 0
    uncertain_count: NonNegativeInt = 0
    latency_ms: NonNegativeFloat | None = None
    success: bool = True
    error_type: str | None = None
    has_more: bool = False
    filled_gap_ids: list[str] = Field(default_factory=list)
    gap_priority_before: NonNegativeFloat = 0.0
    gap_priority_after: NonNegativeFloat = 0.0
    reward: float = 0.0
    metadata: dict[str, Any] = Field(default_factory=dict)
class SourceCheckpoint(FlexibleModel):
    source_name: RegisteredSourceName
    watermark: datetime | None = None
    overlap_hours: NonNegativeInt = 72
    last_attempt_at: datetime | None = None
    last_success_at: datetime | None = None
    complete: bool = False
    cursor: str | None = None
    metadata: dict[str, Any] = Field(default_factory=dict)
class CandidateTopic(FlexibleModel):
    candidate_id: str
    proposed_name: str
    status: Literal["proposed", "confirmed", "rejected"] = "proposed"
    supporting_item_ids: list[str] = Field(default_factory=list)
    source_names: list[str] = Field(default_factory=list)
    mean_relevance: Score = 0.0
    max_anchor_similarity: Score = 0.0
    emergence_score: Score = 0.0
    rationale: str = ""
    metadata: dict[str, Any] = Field(default_factory=dict)
class SearchReflectionDecision(FlexibleModel):
    rewritten_queries: list[SearchQueryPlan] = Field(default_factory=list)
    topics_to_expand: list[str] = Field(default_factory=list)
    topics_to_stop: list[str] = Field(default_factory=list)
    rationale: str = ""
    confidence: Score = 0.0
class SearchCompletenessAssessment(FlexibleModel):
    completeness_score: Score = 0.0
    should_continue: bool = True
    missing_dimensions: list[str] = Field(default_factory=list)
    recommended_next_mode: RunMode | None = None
    stop_rationale: str | None = None
# ---------------------------------------------------------------------
runtime
class ActionDecision(FlexibleModel):
    action_type: str
```

```
    priority: Priority = "medium"
    rationale: str
    expected_gain: str | None = None
    required_context: list[str] = Field(default_factory=list)
    metadata: dict[str, Any] = Field(default_factory=dict)
class ErrorRecord(FlexibleModel):
    error_type: str
    message: str
    retryable: bool = False
    occurred_at: datetime = Field(default_factory=utcnow)
    metadata: dict[str, Any] = Field(default_factory=dict)
class RunBudget(FlexibleModel):
    max_rounds: NonNegativeInt | None = 10
    max_seconds: NonNegativeFloat | None = None
    max_api_calls: NonNegativeInt | None = None
    max_sources: NonNegativeInt | None = None
class RunMetrics(FlexibleModel):
    round_index: NonNegativeInt = 0
    elapsed_seconds: NonNegativeFloat = 0.0
    api_calls_used: NonNegativeInt = 0
    sources_used: NonNegativeInt = 0
class IntelRunBlackboard(FlexibleModel):
    run_id: str
    run_goal: str
    run_mode: RunMode = "bootstrap"
    budget: RunBudget = Field(default_factory=RunBudget)
    metrics: RunMetrics = Field(default_factory=RunMetrics)
    approved_sources: list[ApprovedSource] = Field(default_factory=list)
    query_history: list[QueryHistoryEntry] = Field(default_factory=list)
    raw_items: list[RawIntelItem] = Field(default_factory=list)
    coverage_gaps: list[CoverageGap] = Field(default_factory=list)
    corpus_gaps: list[CorpusGap] = Field(default_factory=list)
    run_gaps: list[RunGap] = Field(default_factory=list)
    query_outcomes: list[QueryCallOutcome] = Field(default_factory=list)
    source_checkpoints: list[SourceCheckpoint] = Field(default_factory=list)
    candidate_topics: list[CandidateTopic] = Field(default_factory=list)
    extended_trends: dict[str, dict[str, float]] =
Field(default_factory=dict)
    sdk_session_id: str | None = None
    stop_reason: str | None = None
    collection_strategy: Literal["adaptive", "legacy"] = "legacy"
    reflection_notes: list[SearchReflectionDecision] =
Field(default_factory=list)
    action_history: list[ActionDecision] = Field(default_factory=list)
    errors: list[ErrorRecord] = Field(default_factory=list)
# ------------------------------------------------------- agent decision
outputs
class TerminationDecision(StrictDecision):
    """Critic verdict on whether to run another collection round."""
    should_continue: bool
    completeness_score: Score = 0.0
    missing_topics: list[str] = Field(default_factory=list)
    stop_reason: str = ""
```

```
    rationale: str = ""
class CandidateTopicNamingDecision(StrictDecision):
    """LLM label for an already deterministically gated residual cluster."""
    proposed_name: str
    rationale: str = ""
#   backend/intel_agent/sources.py
"""Registered intelligence source clients: NVD, arXiv, CISA KEV, OSV.dev.
Self-contained port of the legacy ``registered_source_tools`` HTTP/parse
layer
with the CrewAI ``BaseTool`` wrapper removed (the agentic loop calls these
plain
functions through ``@tool`` wrappers instead) and a small shared retry added
to
the previously bare urllib calls.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
import json
import re
import time
import xml.etree.ElementTree as ET
from datetime import datetime, timezone
from http.client import RemoteDisconnected
from typing import Any
from urllib.error import HTTPError, URLError
from urllib.parse import urlencode
from urllib.request import Request, urlopen
from .schemas import ApprovedSource, RawIntelItem, SourceExecutionStat
from .topics import detect_topics, tokenize
NVD_CVE_API_URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"
ARXIV_API_URL = "https://export.arxiv.org/api/query"
CISA_KEV_JSON_URL = (

"https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilit
ies.json"
)
OSV_QUERY_URL = "https://api.osv.dev/v1/query"
OSV_VULN_URL = "https://api.osv.dev/v1/vulns"
REGISTERED_API_SOURCE_NAMES = {"nvd_cve_api", "arxiv_api", "cisa_kev_json",
"osv_dev_api"}
USER_AGENT = "intel-agent/0.1"
ATOM_NAMESPACE = {"atom": "http://www.w3.org/2005/Atom"}
CVE_PATTERN = re.compile(r"CVE-\d{4}-\d{4,}", re.IGNORECASE)
ADVISORY_ID_PATTERN = re.compile(
    r"(GHSA-[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}|PYSEC-\d{4}-\d+|RUSTSEC-
\d{4}-\d+)",
    re.IGNORECASE,
)
_RETRYABLE = (URLError, TimeoutError, RemoteDisconnected, ConnectionError)
DEFAULT_OSV_PACKAGE_TARGETS = [
    {"ecosystem": "PyPI", "name": "langchain"},
    {"ecosystem": "PyPI", "name": "llama-index"},
    {"ecosystem": "PyPI", "name": "transformers"},
    {"ecosystem": "PyPI", "name": "gradio"},
    {"ecosystem": "PyPI", "name": "mlflow"},
```

```
{"ecosystem": "PyPI", "name": "vllm"},
{"ecosystem": "PyPI", "name": "open-webui"},
```

```
    {"ecosystem": "PyPI", "name": "langflow"},
    {"ecosystem": "PyPI", "name": "chromadb"},
    {"ecosystem": "PyPI", "name": "qdrant-client"},
    {"ecosystem": "PyPI", "name": "weaviate-client"},
    {"ecosystem": "PyPI", "name": "jupyter-server"},
    {"ecosystem": "PyPI", "name": "ray"},
    {"ecosystem": "npm", "name": "flowise"},
    {"ecosystem": "npm", "name": "langchain"},
]
NVD_AI_ATTACK_KEYWORDS = [
    "prompt injection", "indirect prompt injection", "jailbreak",
    "large language model", "LLM", "chatbot", "model extraction",
    "model inversion", "data poisoning", "adversarial example",
    "training data", "RAG", "embedding", "vector database",
]
NVD_AI_PRODUCT_KEYWORDS = [
    "LangChain", "LlamaIndex", "Ollama", "vLLM", "llama.cpp", "Hugging Face",
    "Transformers", "MLflow", "Gradio", "Jupyter", "Ray", "Kubeflow",
    "TensorFlow", "PyTorch", "NVIDIA Triton", "ONNX", "Dify", "Langflow",
    "Flowise", "Milvus", "Qdrant", "Weaviate", "Chroma",
]
NVD_AI_RELEVANT_CWE_IDS = [
    "CWE-20", "CWE-22", "CWE-78", "CWE-79", "CWE-89", "CWE-94",
    "CWE-200", "CWE-287", "CWE-434", "CWE-502", "CWE-918",
]
def default_registered_api_sources() -> list[ApprovedSource]:
    return [
        ApprovedSource(
            source_name="nvd_cve_api",
            base_uri=NVD_CVE_API_URL,
            source_type="security_db",
            trust_level=0.9,
            notes="NVD CVE API 2.0 (keywordSearch, pubStartDate/pubEndDate,
hasKev, cveId)."
,
            metadata={"supports": ["keywordSearch", "pubStartDate",
"pubEndDate", "hasKev"]}
,
        ),
        ApprovedSource(
            source_name="arxiv_api",
            base_uri=ARXIV_API_URL,
            source_type="paper",
            trust_level=0.72,
            notes="arXiv Atom API for LLM security papers and preprints.",
            metadata={"best_for": ["prompt injection", "jailbreak", "rag
poisoning"]},
        ),
        ApprovedSource(
            source_name="cisa_kev_json",
            base_uri=CISA_KEV_JSON_URL,
            source_type="advisory",
            trust_level=0.92,
```

```
            notes="CISA Known Exploited Vulnerabilities JSON catalog.",
            metadata={"best_for": ["known exploited vulnerabilities", "kev"]},
        ),
        ApprovedSource(
            source_name="osv_dev_api",
            base_uri=OSV_QUERY_URL,
            source_type="security_db",
            trust_level=0.84,
            notes="OSV.dev API for open source ecosystem vulnerability
intelligence.",
            metadata={"best_for": ["GitHub Advisory", "PyPI", "RustSec",
"npm", "Go"]},
        ),
    ]
# ---------------------------------------------------------------------
dispatch
def search_source(
    source_name: str,
    query_text: str,
    target_topics: list[str] | None = None,
    max_results: int = 10,
    timeout_seconds: int = 20,
    **params: Any,
) -> tuple[list[RawIntelItem], SourceExecutionStat]:
    """Run one source search; never raises -- failures return an error
stat."""
    start = time.perf_counter()
    topics = target_topics or []
    try:
        if source_name == "nvd_cve_api":
            items = search_nvd_cve_api(
                query_text, max_results, timeout_seconds,
                keyword_search=params.get("nvd_keyword_search"),
                pub_start_date=params.get("nvd_pub_start_date"),
                pub_end_date=params.get("nvd_pub_end_date"),
                has_kev=bool(params.get("nvd_has_kev")),
                cve_id=params.get("nvd_cve_id"),

keyword_exact_match=bool(params.get("nvd_keyword_exact_match")),
                cwe_id=params.get("nvd_cwe_id"),
                cvss_v3_severity=params.get("nvd_cvss_v3_severity"),
                no_rejected=bool(params.get("nvd_no_rejected", True)),
                start_index=int(params.get("nvd_start_index") or 0),
            )
        elif source_name == "arxiv_api":
            items = search_arxiv_api(
                query_text, topics, max_results, timeout_seconds,
                arxiv_search_query=params.get("arxiv_search_query"),
                arxiv_start=int(params.get("arxiv_start") or 0),
            )
        elif source_name == "cisa_kev_json":
            items = search_cisa_kev_json(
                query_text, max_results, timeout_seconds,
                cve_ids=params.get("cisa_cve_ids") or [],
                keyword=params.get("cisa_keyword"),
```

```
            )
        elif source_name == "osv_dev_api":
            items = search_osv_dev_api(
                query_text, max_results, timeout_seconds,
                ecosystem=params.get("osv_ecosystem"),
                package_name=params.get("osv_package_name"),
                version=params.get("osv_version"),
                purl=params.get("osv_purl"),
                vuln_id=params.get("osv_vuln_id"),
            )
        else:
            items = []
        stat = SourceExecutionStat(
            source_name=source_name, query_count=1, result_count=len(items),
            success=True, latency_ms=_elapsed_ms(start), notes="ok",
        )
        return items, stat
    except (HTTPError, URLError, TimeoutError, RemoteDisconnected,
ConnectionError,
            ValueError, ET.ParseError) as exc:
        stat = SourceExecutionStat(
            source_name=source_name, query_count=1, result_count=0,
success=False,
            latency_ms=_elapsed_ms(start), error_type=exc.__class__.__name__,
notes=str(exc)
,
        )
        return [], stat
# ---------------------------------------------------------------- per-
source
def search_nvd_cve_api(
    query_text: str, max_results: int, timeout_seconds: int,
    keyword_search: str | None = None, pub_start_date: str | None = None,
    pub_end_date: str | None = None, has_kev: bool = False, cve_id: str |
None = None,
    keyword_exact_match: bool = False, cwe_id: str | None = None,
    cvss_v3_severity: str | None = None, no_rejected: bool = True,
    start_index: int = 0,
) -> list[RawIntelItem]:
    params = _build_nvd_query_params(
        query_text, max_results, keyword_search, pub_start_date, pub_end_date,
        has_kev, cve_id, keyword_exact_match, cwe_id, cvss_v3_severity,
no_rejected,
        start_index,
    )
    data = _fetch_json(_url_with_params(NVD_CVE_API_URL, params),
timeout_seconds)
    return parse_nvd_items(data)
def search_arxiv_api(
    query_text: str, target_topics: list[str], max_results: int,
    timeout_seconds: int, arxiv_search_query: str | None = None,
    arxiv_start: int = 0,
) -> list[RawIntelItem]:
    search_query = arxiv_search_query or build_arxiv_search_query(query_text,
target_topics)
```

```
   params = {
       "search_query": search_query, "start": str(max(0, arxiv_start)),
"max_results": str(
max_results),
```

```
        "sortBy": "submittedDate", "sortOrder": "descending",
    }
    raw_xml = _fetch_text(_url_with_params(ARXIV_API_URL, params),
timeout_seconds)
    return parse_arxiv_items(raw_xml)
def search_cisa_kev_json(
    query_text: str, max_results: int, timeout_seconds: int,
    cve_ids: list[str] | None = None, keyword: str | None = None,
) -> list[RawIntelItem]:
    data = _fetch_json(CISA_KEV_JSON_URL, timeout_seconds)
    return parse_cisa_kev_items(data, query_text, cve_ids or [], keyword,
max_results)
def search_osv_dev_api(
    query_text: str, max_results: int, timeout_seconds: int,
    ecosystem: str | None = None, package_name: str | None = None,
    version: str | None = None, purl: str | None = None, vuln_id: str | None
= None,
) -> list[RawIntelItem]:
    direct_id = vuln_id or _extract_first_vulnerability_id(query_text)
    if direct_id:
        data = _fetch_json(f"{OSV_VULN_URL}/{direct_id}", timeout_seconds)
        return parse_osv_vulnerability(data)[:max_results]
    payload = build_osv_query_payload(ecosystem, package_name, version, purl)
    payloads = [payload] if payload else [{"package": pkg} for pkg in
DEFAULT_OSV_PACKAGE_TA
RGETS]
    items: list[RawIntelItem] = []
    seen: set[str] = set()
    for next_payload in payloads:
        data = _fetch_json(OSV_QUERY_URL, timeout_seconds, method="POST",
payload=next_paylo
ad)
        for item in parse_osv_query_response(data, query_text):
            if item.item_id in seen:
                continue
            seen.add(item.item_id)
            items.append(item)
            if len(items) >= max_results:
                return items
    return items
# ------------------------------------------------------------ query
builders
def _build_nvd_query_params(
    query_text: str, max_results: int, keyword_search: str | None,
    pub_start_date: str | None, pub_end_date: str | None, has_kev: bool,
    cve_id: str | None, keyword_exact_match: bool, cwe_id: str | None,
    cvss_v3_severity: str | None, no_rejected: bool,
    start_index: int = 0,
) -> dict[str, str | None]:
    cve = cve_id or _extract_first_cve_id(query_text)
    params: dict[str, str | None] = {
        "resultsPerPage": str(max_results), "startIndex": str(max(0,
start_index))
    }
    if cve:
```

```
    params["cveId"] = cve
else:
```

```
        params["keywordSearch"] = keyword_search or
_compact_keyword_query(query_text)
        if keyword_exact_match and " " in params["keywordSearch"]:
            params["keywordExactMatch"] = None
    if cwe_id:
        params["cweId"] = cwe_id
    if cvss_v3_severity:
        params["cvssV3Severity"] = cvss_v3_severity.upper()
    if pub_start_date and pub_end_date:
        params["pubStartDate"] = pub_start_date
        params["pubEndDate"] = pub_end_date
    if has_kev:
        params["hasKev"] = None
    if no_rejected:
        params["noRejected"] = None
    return params
def build_arxiv_search_query(query_text: str, target_topics: list[str]) ->
str:
    phrases = target_topics or detect_topics(query_text) or [
        "prompt injection", "jailbreak", "RAG poisoning", "large language
model security",
    ]
    phrase_query = " OR ".join(f'all:"{phrase}"' for phrase in phrases[:6])
    return f"({phrase_query}) AND (cat:cs.CR OR cat:cs.AI OR cat:cs.CL)"
def build_osv_query_payload(
    ecosystem: str | None = None, package_name: str | None = None,
    version: str | None = None, purl: str | None = None,
) -> dict[str, Any]:
    if purl:
        payload: dict[str, Any] = {"package": {"purl": purl}}
    elif ecosystem and package_name:
        payload = {"package": {"ecosystem": ecosystem, "name": package_name}}
    else:
        return {}
    if version and "@" not in payload["package"].get("purl", ""):
        payload["version"] = version
    return payload
# ---------------------------------------------------------------------
parsers
def parse_nvd_items(data: dict[str, Any]) -> list[RawIntelItem]:
    items: list[RawIntelItem] = []
    for entry in data.get("vulnerabilities", []):
        cve = entry.get("cve", {})
        cve_id = cve.get("id", "")
        if not cve_id:
            continue
        description = _first_english_description(cve.get("descriptions", []))
        references = _extract_nvd_references(cve)
        primary_ref = references[0].get("url") if references else
f"{NVD_CVE_API_URL}?cveId=
{cve_id}"
        cwe_ids = _extract_nvd_cwe_ids(cve)
        cpe_products = _extract_nvd_cpe_products(cve)
        nvd_text = " ".join([description, " ".join(cpe_products)])
        topics = detect_topics(nvd_text)
```

```
       ai_category, matched_terms = classify_ai_vulnerability(nvd_text,
cwe_ids)
       relevance = estimate_relevance(nvd_text, topics)
       if ai_category != "uncategorized":
          relevance = max(relevance, 0.68)
       items.append(
          RawIntelItem(
             item_id=f"nvd:{cve_id.lower()}",
             source_name="nvd_cve_api",
             source_uri=primary_ref,
             title=f"{cve_id}: {_shorten(description, 120)}",
             summary=description,
             published_at=_parse_datetime(cve.get("published")),
             raw_text=description,
             relevance_score=relevance,
             extraction_notes="Parsed from NVD CVE API 2.0 response.",
             metadata={
                "cve_id": cve_id, "topics": topics, "ai_intel_category":
ai_category,
                "matched_ai_terms": matched_terms, "cwe_ids": cwe_ids,
                "cvss_v3_severity": _extract_nvd_cvss_v3_severity(cve),
                "cpe_products": cpe_products, "source_type": "security_db",
             },
          )
       )
    return items
def parse_arxiv_items(raw_xml: str) -> list[RawIntelItem]:
    root = ET.fromstring(raw_xml)
    items: list[RawIntelItem] = []
    for entry in root.findall("atom:entry", ATOM_NAMESPACE):
       item_id = _node_text(entry, "atom:id")
       title = _clean_text(_node_text(entry, "atom:title"))
       summary = _clean_text(_node_text(entry, "atom:summary"))
       if title.lower() == "error":
          continue
       topics = detect_topics(f"{title} {summary}")
       items.append(
          RawIntelItem(
             item_id=f"arxiv:{item_id.rsplit('/', 1)[-1].lower()}",
             source_name="arxiv_api",
             source_uri=item_id,
             title=title,
             summary=summary,
             published_at=_parse_datetime(_node_text(entry,
"atom:published")),
             raw_text=summary,
             relevance_score=estimate_relevance(f"{title} {summary}",
topics),
             extraction_notes="Parsed from arXiv Atom API response.",
             metadata={
                "topics": topics, "source_type": "paper",
                "authors": [
                   _node_text(author, "atom:name")
                   for author in entry.findall("atom:author",
ATOM_NAMESPACE)
```

```
                ],
            },
        )
    )
    return items
def parse_cisa_kev_items(
    data: dict[str, Any], query_text: str, cve_ids: list[str],
    keyword: str | None, max_results: int,
) -> list[RawIntelItem]:
    wanted = {cve_id.upper() for cve_id in cve_ids}
    wanted.update(match.upper() for match in CVE_PATTERN.findall(query_text))
    query_tokens = tokenize(keyword or query_text)
    items: list[RawIntelItem] = []
    for entry in data.get("vulnerabilities", []):
        cve_id = str(entry.get("cveID", "")).upper()
        text = " ".join(
            str(entry.get(key, ""))
            for key in ("vendorProject", "product", "vulnerabilityName",
                        "shortDescription", "requiredAction",
"knownRansomwareCampaignUse")
        )
        if wanted and cve_id not in wanted:
            continue
        if not wanted and query_tokens and not (query_tokens &
tokenize(text)):
            continue
        topics = detect_topics(text)
        items.append(
            RawIntelItem(
                item_id=f"cisa-kev:{cve_id.lower()}",
                source_name="cisa_kev_json",
                source_uri=(
                    "https://www.cisa.gov/known-exploited-vulnerabilities-
catalog"
                    f"?search_api_fulltext={cve_id}"
                ),
                title=f"{cve_id}: {entry.get('vulnerabilityName', '')}",
                summary=str(entry.get("shortDescription", "")),
                published_at=_parse_date(entry.get("dateAdded")),
                raw_text=text,
                relevance_score=max(0.55, estimate_relevance(text, topics)),
                extraction_notes="Parsed from CISA KEV JSON catalog.",
                metadata={
                    "cve_id": cve_id, "topics": topics,
                    "vendor_project": entry.get("vendorProject"),
                    "product": entry.get("product"), "due_date":
entry.get("dueDate"),
                    "date_added": entry.get("dateAdded"), "source_type":
"advisory",
                },
            )
        )
        if len(items) >= max_results:
            break
    return items
```

```
def parse_osv_query_response(data: dict[str, Any], query_text: str) ->
list[RawIntelItem]:
    items: list[RawIntelItem] = []
    for vuln in data.get("vulns", []):
        items.extend(parse_osv_vulnerability(vuln, query_text))
    return items
def parse_osv_vulnerability(data: dict[str, Any], query_text: str = "") ->
list[RawIntelItem
]:
    vuln_id = str(data.get("id", ""))
    if not vuln_id:
        return []
    summary = str(data.get("summary") or data.get("details") or "")
    details = str(data.get("details") or summary)
    affected = data.get("affected", [])
    references = data.get("references", [])
    source_uri = references[0].get("url") if references else
f"{OSV_VULN_URL}/{vuln_id}"
    topics = detect_topics(f"{summary} {details} {query_text}")
    ecosystems = sorted(
        {
            pkg.get("ecosystem", "")
            for item in affected
            for pkg in [item.get("package", {})]
            if pkg.get("ecosystem")
        }
    )
    return [
        RawIntelItem(
            item_id=f"osv:{vuln_id.lower()}",
            source_name="osv_dev_api",
            source_uri=source_uri,
            title=f"{vuln_id}: {_shorten(summary, 160)}",
            summary=summary or _shorten(details, 300),
            published_at=_parse_datetime(data.get("published") or
data.get("modified")),
            raw_text=details,
            relevance_score=estimate_relevance(f"{summary} {details}
{query_text}", topics),
            extraction_notes="Parsed from OSV.dev vulnerability API
response.",
            metadata={
                "vulnerability_id": vuln_id, "aliases": data.get("aliases",
[]),
                "topics": topics, "ecosystems": ecosystems,
                "modified": data.get("modified"), "source_type": "security_db",
            },
        )
    ]
# ----------------------------------------------------------------------- http
def _fetch_json(
    url: str, timeout_seconds: int, method: str = "GET",
    payload: dict[str, Any] | None = None, retries: int = 2,
) -> dict[str, Any]:
    data = None
```

```
    headers = {"Accept": "application/json", "User-Agent": USER_AGENT}
    if payload is not None:
```

```
        data = json.dumps(payload).encode("utf-8")
        headers["Content-Type"] = "application/json"
    nvd_key = _setting("NVD_API_KEY")
    if nvd_key and url.startswith(NVD_CVE_API_URL):
        headers["apiKey"] = nvd_key
    raw = _request_with_retry(url, headers, data, method, timeout_seconds,
retries)
    return json.loads(raw)
def _fetch_text(url: str, timeout_seconds: int, retries: int = 2) -> str:
    return _request_with_retry(
        url, {"User-Agent": USER_AGENT}, None, "GET", timeout_seconds, retries
    )
def _request_with_retry(
    url: str, headers: dict[str, str], data: bytes | None,
    method: str, timeout_seconds: int, retries: int,
) -> str:
    last_exc: Exception | None = None
    for attempt in range(retries + 1):
        try:
            request = Request(url, data=data, headers=headers, method=method)
            with urlopen(request, timeout=timeout_seconds) as response:
                return response.read().decode("utf-8")
        except HTTPError as exc:
            if exc.code in (429, 500, 502, 503, 504) and attempt < retries:
                last_exc = exc
                time.sleep(1.5 * (attempt + 1))
                continue
            raise
        except _RETRYABLE as exc:
            last_exc = exc
            if attempt < retries:
                time.sleep(1.5 * (attempt + 1))
                continue
            raise
    raise last_exc if last_exc else RuntimeError("request failed without
exception")
def _url_with_params(base_url: str, params: dict[str, str | None]) -> str:
    parts = [key if value is None else urlencode({key: value}) for key, value
in params.item
s()]
    return f"{base_url}?{'&'.join(parts)}"
# ------------------------------------------------------------------ helpers
def estimate_relevance(text: str, topics: list[str]) -> float:
    topic_bonus = min(0.3, 0.1 * len(topics))
    token_bonus = min(0.2, 0.02 * len(tokenize(text)))
    return min(1.0, 0.45 + topic_bonus + token_bonus)
def classify_ai_vulnerability(text: str, cwe_ids: list[str]) -> tuple[str,
list[str]]:
    lower = text.lower()
    attack = [term for term in NVD_AI_ATTACK_KEYWORDS if term.lower() in
lower]
    product = [term for term in NVD_AI_PRODUCT_KEYWORDS if term.lower() in
lower]
    cwe = [cwe_id for cwe_id in cwe_ids if cwe_id in NVD_AI_RELEVANT_CWE_IDS]
    matched = sorted(set(attack + product + cwe))
```

```
if attack:
```

```
        return "ai_native_attack", matched
    if product and cwe:
        return "ai_application_infrastructure_vulnerability", matched
    if product:
        return "ai_data_model_supply_chain", matched
    return "uncategorized", matched
def _compact_keyword_query(query_text: str) -> str:
    topics = detect_topics(query_text)
    if topics:
        return " ".join(topics[:3])
    tokens = sorted(tokenize(query_text))
    return " ".join(tokens[:4]) or "large language model"
def _extract_first_vulnerability_id(query_text: str) -> str | None:
    cve_id = _extract_first_cve_id(query_text)
    if cve_id:
        return cve_id
    match = ADVISORY_ID_PATTERN.search(query_text)
    return match.group(0).upper() if match else None
def _extract_first_cve_id(query_text: str) -> str | None:
    match = CVE_PATTERN.search(query_text)
    return match.group(0).upper() if match else None
def _extract_nvd_cwe_ids(cve: dict[str, Any]) -> list[str]:
    cwe_ids: set[str] = set()
    for weakness in cve.get("weaknesses", []):
        for description in weakness.get("description", []):
            value = str(description.get("value", ""))
            if value.startswith("CWE-"):
                cwe_ids.add(value)
    return sorted(cwe_ids)
def _extract_nvd_references(cve: dict[str, Any]) -> list[dict[str, Any]]:
    raw = cve.get("references", [])
    if isinstance(raw, dict):
        data = raw.get("referenceData", [])
        return data if isinstance(data, list) else []
    if isinstance(raw, list):
        return [item for item in raw if isinstance(item, dict)]
    return []
def _extract_nvd_cvss_v3_severity(cve: dict[str, Any]) -> str | None:
    metrics = cve.get("metrics", {})
    for key in ("cvssMetricV31", "cvssMetricV30"):
        values = metrics.get(key) or []
        if values:
            severity = values[0].get("cvssData", {}).get("baseSeverity")
            if severity:
                return str(severity)
    return None
def _extract_nvd_cpe_products(cve: dict[str, Any]) -> list[str]:
    products: set[str] = set()
    for configuration in cve.get("configurations", []):
        for node in configuration.get("nodes", []):
```

```
          for match in node.get("cpeMatch", []):
             parts = str(match.get("criteria", "")).split(":")
             if len(parts) >= 6:
                vendor = parts[3].replace("_", " ")
                product = parts[4].replace("_", " ")
                products.add(f"{vendor} {product}".strip())
    return sorted(products)
def _first_english_description(descriptions: list[dict[str, Any]]) -> str:
    for description in descriptions:
        if description.get("lang") == "en":
            return str(description.get("value", ""))
    return str(descriptions[0].get("value", "")) if descriptions else ""
def _parse_datetime(value: Any) -> datetime | None:
    if not value:
        return None
    raw = str(value).replace("Z", "+00:00")
    try:
        parsed = datetime.fromisoformat(raw)
    except ValueError:
        return _parse_date(raw)
    return parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None else
parsed.astimezo
ne(timezone.utc)
def _parse_date(value: Any) -> datetime | None:
    if not value:
        return None
    try:
        return datetime.strptime(str(value), "%Y-%m-
%d").replace(tzinfo=timezone.utc)
    except ValueError:
        return None
def _node_text(node: ET.Element, path: str) -> str:
    found = node.find(path, ATOM_NAMESPACE)
    return found.text if found is not None and found.text is not None else ""
def _clean_text(text: str) -> str:
    return re.sub(r"\s+", " ", text).strip()
def _shorten(text: str, max_length: int) -> str:
    cleaned = _clean_text(text)
    return cleaned if len(cleaned) <= max_length else f"{cleaned[: max_length
- 3].rstrip()}
..."
def _elapsed_ms(start: float) -> float:
    return round((time.perf_counter() - start) * 1000, 2)
#   backend/intel_agent/store.py
"""Store selection: MongoDB when configured, JSON files otherwise.
``default_store()`` returns a :class:`MongoIntelRunStore` when Mongo is
enabled
(``INTEL_MONGO_URI`` set or ``INTEL_MONGO_ENABLED`` truthy) *and*
``pymongo`` is
importable; otherwise it falls back to the dependency-free
:class:`JsonIntelRunStore` so offline runs and tests need no database.
"""
from __future__ import annotations
from typing import Any
from .mongo_store import mongo_enabled
```

```
from .persistence import JsonIntelRunStore
def default_store() -> Any:
    if mongo_enabled():
        try:
            import pymongo  # noqa: F401
        except ImportError:
            print("[warn] INTEL_MONGO_* set but pymongo is not installed;
using JSON store."
)
            return JsonIntelRunStore()
        from .mongo_store import MongoIntelRunStore
        return MongoIntelRunStore()
    return JsonIntelRunStore()
#   backend/intel_agent/tools/__init__.py
"""In-process MCP tools the collection agent calls during a round."""
from __future__ import annotations
__all__ = ["ToolContext", "build_round_server"]
def __getattr__(name: str):
    if name == "ToolContext":
        from .context import ToolContext
        return ToolContext
    if name == "build_round_server":
        from .server import build_round_server
        return build_round_server
    raise AttributeError(name)
#   backend/intel_agent/tools/adaptive_tools.py
"""Planning tools that keep LLM proposals behind deterministic scoring."""
from __future__ import annotations
import json
from typing import Any
from pydantic import ValidationError
from ..engine.bandit import select_portfolio
from ..engine.gaps import SOURCE_CHANNEL
from ..schemas import QueryCandidate
from ..topics import ALL_SECURITY_TOPICS
from .context import ToolContext
_STATE_SCHEMA = {"type": "object", "properties": {}}
_CANDIDATE_SCHEMA = {
    "type": "object",
    "properties": {
        "candidates": {
            "type": "array",
            "maxItems": 8,
            "items": {
                "type": "object",
                "properties": {
                    "candidate_id": {"type": "string"},
                    "source_name": {
                        "type": "string",
                        "enum": ["nvd_cve_api", "arxiv_api", "cisa_kev_json",
"osv_dev_api"]
,
```

```
                },
                "query_text": {"type": "string"},
                "params": {
                   "type": "object",
                   "description": (
                      "All source-specific operators must be nested here,
e.g. "
                      "arxiv_search_query, nvd_cwe_id, osv_ecosystem, or
cisa_keyword.
"
                   ),
                },
                "query_intent": {
                   "type": "string",
                   "enum": ["gap_fill", "discovery", "entity_expand",
"confirmation"],
                },
                "evidence_channel": {
                   "type": "string",
                   "enum": ["research", "vulnerability_advisory",
"exploitation"],
                },
                "target_topics": {"type": "array", "items": {"type":
"string"}},
                "target_gap_ids": {"type": "array", "items": {"type":
"string"}},
                "rationale": {"type": "string"},
             },
             "required": [
                "candidate_id", "source_name", "query_text", "query_intent",
                "evidence_channel", "target_topics", "target_gap_ids",
             ],
          },
       }
   },
   "required": ["candidates"],
}
def build_adaptive_tools(ctx: ToolContext) -> list[Any]:
   from claude_agent_sdk import tool
   @tool(
      "get_collection_state",
      "Return the authoritative Core CorpusGaps, current RunGaps, source
checkpoints, and
"
      "recent per-call rewards. Extended topics never appear as coverage
gaps.",
      _STATE_SCHEMA,
   )
   async def get_collection_state(args: dict[str, Any]) -> dict[str, Any]:
      state = {
         "corpus_gaps": [
            gap.model_dump(mode="json")
            for gap in ctx.corpus_gaps
            if gap.status == "open"
         ][:24],
```

```
          "run_gaps": [gap.model_dump(mode="json") for gap in ctx.run_gaps
if gap.status =
= "open"],
          "source_checkpoints": [checkpoint.model_dump(mode="json") for
checkpoint in ctx.
```

```
source_checkpoints],
          "extended_trends": ctx.extended_trends,
          "historical_reward_summary": _historical_reward_summary(ctx),
          "api_calls_remaining": (
             None if ctx.max_api_calls is None else max(0, ctx.max_api_calls
- ctx.api_ca
lls_used)
          ),
       }
       return _text(json.dumps(state, ensure_ascii=False))
   @tool(
       "score_query_candidates",
       "Submit 1-8 source-specific query candidates. Deterministic UCB
scoring approves at
"
       "most four. Only approved candidate IDs may be passed to source
tools.",
       _CANDIDATE_SCHEMA,
   )
   async def score_query_candidates(args: dict[str, Any]) -> dict[str, Any]:
       raw = list(args.get("candidates") or [])[:8]
       candidates: list[QueryCandidate] = []
       errors: list[str] = []
       seen: set[str] = set()
       for entry in raw:
          try:
             candidate = QueryCandidate.model_validate(entry)
          except ValidationError as exc:
             errors.append(str(exc.errors(include_url=False))[:300])
             continue
          if candidate.candidate_id in seen:
             errors.append(f"duplicate candidate_id:
{candidate.candidate_id}")
             continue
          validation_error = candidate_validation_error(candidate, ctx)
          if validation_error:
             errors.append(f"{candidate.candidate_id}: {validation_error}")
             continue
          seen.add(candidate.candidate_id)
          candidates.append(candidate)
       remaining = (
          ctx.max_calls_this_round
          if ctx.max_api_calls is None
          else min(ctx.max_calls_this_round, max(0, ctx.max_api_calls -
ctx.api_calls_used
))
       )
       selected = select_portfolio(
          candidates,
          ctx.prior_outcomes,
          ctx.corpus_gaps,
          max_calls=remaining,
          max_total_calls=ctx.max_api_calls,
          discovery_unbounded=ctx.extended_focus,
          budget_outcomes=ctx.current_run_outcomes,
```

```
        )
        ctx.approved_candidates = {candidate.candidate_id: candidate for
candidate, _ in sel
ected}
        ctx.candidate_utilities = {candidate.candidate_id: utility for
candidate, utility in
 selected}
        payload = {
            "approved": [
                {**candidate.model_dump(mode="json"), "utility": utility}
                for candidate, utility in selected
            ],
            "rejected_count": len(candidates) - len(selected),
            "validation_errors": errors,
        }
        return _text(json.dumps(payload, ensure_ascii=False))
    return [get_collection_state, score_query_candidates]
def candidate_validation_error(
    candidate: QueryCandidate, ctx: ToolContext
) -> str | None:
    from .source_tools import candidate_exceeds_time_scope
    if candidate.evidence_channel != SOURCE_CHANNEL[candidate.source_name]:
        return "source and evidence_channel do not match"
    unknown_topics = sorted(set(candidate.target_topics) -
set(ALL_SECURITY_TOPICS))
    if unknown_topics:
        return f"unknown taxonomy topics: {unknown_topics}"
    open_gap_ids = {
        gap.gap_id for gap in ctx.corpus_gaps if gap.status == "open"
    } | {gap.gap_id for gap in ctx.run_gaps if gap.status == "open"}
    invalid_gap_ids = sorted(set(candidate.target_gap_ids) - open_gap_ids)
    if invalid_gap_ids:
        return f"target gaps are not open: {invalid_gap_ids}"
    if candidate_exceeds_time_scope(
        ctx, candidate.source_name, candidate.params
    ):
        return "candidate time operators exceed the authoritative incremental
window"
    if candidate.query_intent == "gap_fill" and not candidate.target_gap_ids:
        return "gap_fill candidates require an open target gap"
    return None
def _historical_reward_summary(ctx: ToolContext) -> list[dict[str, Any]]:
    arms: dict[tuple[str, str, str, str, str], dict[str, float]] = {}
    for outcome in ctx.prior_outcomes:
        topic = outcome.target_topics[0] if outcome.target_topics else
"general"
        key = (
            topic,
            outcome.evidence_channel,
            outcome.source_name,
            outcome.query_intent,
            outcome.operator_signature,
        )
        row = arms.setdefault(
            key,
```

```
            {"calls": 0.0, "reward": 0.0, "noise": 0.0, "duplicate": 0.0},
        )
        row["calls"] += 1
        row["reward"] += outcome.reward
        row["noise"] += outcome.noise_count / max(1, outcome.new_count)
        row["duplicate"] += outcome.duplicate_count / max(1,
outcome.in_scope_count)
    summary = []
    for key, totals in arms.items():
        calls = int(totals["calls"])
        summary.append(
            {
                "arm": key,
                "calls": calls,
                "mean_reward": round(totals["reward"] / calls, 4),
                "mean_noise_ratio": round(totals["noise"] / calls, 4),
                "mean_duplicate_ratio": round(totals["duplicate"] / calls, 4),
            }
        )
    return sorted(
        summary,
        key=lambda row: (-row["mean_reward"], -row["calls"], row["arm"]),
    )[:12]
def _text(message: str) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": message}]}
#   backend/intel_agent/tools/constants.py
"""Shared in-process MCP naming constants without tool-construction
imports."""
SERVER_NAME = "intel_sources"
#   backend/intel_agent/tools/context.py
"""Shared per-round state the agent's in-process tools read and write.
One ``ToolContext`` is created per round. The source tools append collected
items, record executed calls (for dedup, budget, and playbook harvesting),
and
enforce the API-call budget and cross-round query dedup directly -- the
reliable
in-process control plane described in the roadmap. Hooks add an auditing
layer
on top but the context is authoritative.
"""
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any
from ..schemas import (
    CorpusGap,
    QueryCallOutcome,
    QueryCandidate,
    RawIntelItem,
    RunGap,
    SourceCheckpoint,
    SourceExecutionStat,
)
@dataclass
class ToolContext:
```

```
    run_id: str
    target_topics: list[str]
    topic_bucket: str
    max_results_per_call: int = 20
    since: datetime | None = None
    until: datetime | None = None
    source_since: dict[str, datetime] = field(default_factory=dict)
    existing_item_ids: set[str] = field(default_factory=set)
    executed_keys: set[str] = field(default_factory=set)
    api_calls_used: int = 0
    max_api_calls: int | None = None
    playbook: Any | None = None  # PlaybookStore | None (avoid import cycle)
    adaptive: bool = False
    corpus_gaps: list[CorpusGap] = field(default_factory=list)
    run_gaps: list[RunGap] = field(default_factory=list)
    prior_outcomes: list[QueryCallOutcome] = field(default_factory=list)
    current_run_outcomes: list[QueryCallOutcome] =
field(default_factory=list)
    source_checkpoints: list[SourceCheckpoint] = field(default_factory=list)
    extended_trends: dict[str, dict[str, float]] =
field(default_factory=dict)
    approved_candidates: dict[str, QueryCandidate] =
field(default_factory=dict)
    candidate_utilities: dict[str, float] = field(default_factory=dict)
    max_calls_this_round: int = 4
    extended_focus: bool = False
    # per-round outputs
    collected_items: dict[str, RawIntelItem] = field(default_factory=dict)
    stats: list[SourceExecutionStat] = field(default_factory=list)
    executed_calls: list[dict[str, Any]] = field(default_factory=list)
    recorded_techniques: list[dict[str, Any]] = field(default_factory=list)
    notes: list[str] = field(default_factory=list)
    def budget_remaining(self) -> bool:
        return self.max_api_calls is None or self.api_calls_used <
self.max_api_calls
    def time_scope_hint(self) -> str | None:
        if self.since is None:
            return None
        if self.until is not None:
            return f"{self.since.date().isoformat()}..
{self.until.date().isoformat()}"
        return f"since {self.since.date().isoformat()}"
    def in_time_scope(self, item: RawIntelItem) -> bool:
        since = self.source_since.get(item.source_name, self.since)
        if since is None:
            return True
        published = item.published_at
        if published is None:
            return True  # keep undated items; the source-side filter already
narrowed
        if published < since:
            return False
        if self.until is not None and published > self.until:
            return False
        return True
```

```
   def source_time_scope(self, source_name: str) -> tuple[datetime | None,
datetime | None]
```

```
:
       return self.source_since.get(source_name, self.since), self.until
#   backend/intel_agent/tools/control_tools.py
"""Round control tool: the agent's explicit exit from a collection round."""
from __future__ import annotations
from typing import Any
from .context import ToolContext
_SUMMARY_SCHEMA = {
   "type": "object",
   "properties": {
      "summary": {"type": "string", "description": "1-3 sentences on what
this round colle
cted."},
      "covered_topics": {
         "type": "array", "items": {"type": "string"},
         "description": "Target topics you believe are now well covered.",
      },
      "remaining_gaps": {
         "type": "array", "items": {"type": "string"},
         "description": "Target topics still thin or unaddressed.",
      },
   },
   "required": ["summary"],
}
def build_control_tools(ctx: ToolContext) -> list[Any]:
   from claude_agent_sdk import tool
   @tool("submit_round_summary", "Call this once when you have finished
searching for this
round. "
        "It ends your turn; the controller then merges results and decides
whether to cont
inue.",
        _SUMMARY_SCHEMA)
   async def submit_round_summary(args: dict[str, Any]) -> dict[str, Any]:
      ctx.notes.append(
         {
            "summary": str(args.get("summary") or "").strip(),
            "covered_topics": list(args.get("covered_topics") or []),
            "remaining_gaps": list(args.get("remaining_gaps") or []),
         }
      )
      return _text("Round summary recorded. You may stop now.")
   return [submit_round_summary]
def _text(message: str) -> dict[str, Any]:
   return {"content": [{"type": "text", "text": message}]}
#   backend/intel_agent/tools/memory_tools.py
"""Playbook tools: let the agent recall past techniques and record new
ones."""
from __future__ import annotations
from typing import Any
from .context import ToolContext
_RECALL_SCHEMA = {
   "type": "object",
   "properties": {
```

```
       "query": {"type": "string", "description": "What you are trying to
find right now."}
,
       "source_name": {"type": "string", "description": "Optional: restrict
to one source."
},
   },
   "required": ["query"],
}
_RECORD_SCHEMA = {
   "type": "object",
   "properties": {
       "technique_text": {"type": "string", "description": "The reusable
search technique,
in plain language."},
       "source_name": {"type": "string", "description": "Source the technique
applies to."}
,
       "reason": {"type": "string", "description": "Why it worked (what it
found, why low-n
oise)."},
   },
   "required": ["technique_text", "source_name"],
}
def build_memory_tools(ctx: ToolContext, *, include_record: bool = True) ->
list[Any]:
   from claude_agent_sdk import tool
   @tool("recall_playbook", "Recall proven search techniques from past runs
before planning
 "
        "queries. Returns techniques ranked by semantic match and historical
payoff.", _RE
CALL_SCHEMA)
   async def recall_playbook(args: dict[str, Any]) -> dict[str, Any]:
       if ctx.playbook is None:
           return _text("Playbook memory is disabled for this run.")
       entries = ctx.playbook.recall(
           query_text=str(args.get("query") or ""),
           topic_bucket=ctx.topic_bucket,
           source_name=args.get("source_name"),
           top_k=5,
       )
       rendered = ctx.playbook.render_recall(entries)
       return _text(rendered or "No matching techniques recorded yet.")
   @tool("record_technique", "Record a search technique that just worked so
future runs can
 reuse "
        "it. Only record techniques grounded in this round's observed
results.", _RECORD_S
CHEMA)
   async def record_technique(args: dict[str, Any]) -> dict[str, Any]:
       ctx.recorded_techniques.append(
           {
               "technique_text": str(args.get("technique_text") or
"").strip(),
```

```
            "source_name": str(args.get("source_name") or "").strip(),
            "reason": str(args.get("reason") or "").strip(),
        }
      )
      return _text("Technique noted; it will be scored against this round's
yield.")
   return (
```

```
        [recall_playbook, record_technique]
        if include_record
        else [recall_playbook]
    )
def _text(message: str) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": message}]}
#   backend/intel_agent/tools/server.py
"""Bundle the per-round tools into one in-process MCP server."""
from __future__ import annotations
from typing import Any
from .constants import SERVER_NAME
from .context import ToolContext
from .control_tools import build_control_tools
from .adaptive_tools import build_adaptive_tools
from .memory_tools import build_memory_tools
from .source_tools import build_source_tools
# Fully-qualified tool names for ClaudeAgentOptions.allowed_tools.
ALLOWED_TOOLS = [
    f"mcp__{SERVER_NAME}__{name}"
    for name in (
        "search_nvd", "search_arxiv", "search_cisa_kev", "search_osv",
        "recall_playbook", "record_technique", "submit_round_summary",
        "get_collection_state", "score_query_candidates",
    )
]
def build_round_server(ctx: ToolContext) -> tuple[dict[str, Any],
list[str]]:
    """Return ({server_name: server_config}, allowed_tools) for a round."""
    from claude_agent_sdk import create_sdk_mcp_server
    tools = (
        build_source_tools(ctx)
        + build_memory_tools(ctx, include_record=not ctx.adaptive)
        + build_control_tools(ctx)
    )
    if ctx.adaptive:
        tools += build_adaptive_tools(ctx)
    server = create_sdk_mcp_server(name=SERVER_NAME, tools=tools)
    allowed = list(ALLOWED_TOOLS)
    if not ctx.adaptive:
        allowed = [
            name
            for name in allowed
            if not name.endswith("__get_collection_state")
            and not name.endswith("__score_query_candidates")
        ]
    else:
        allowed = [name for name in allowed if not
name.endswith("__record_technique")]
    return {SERVER_NAME: server}, allowed
#   backend/intel_agent/tools/source_tools.py
"""Source-search tools the agent calls directly inside a round.
Each registered source is a typed ``@tool`` exposing its advanced operators,
so
```

```
the agent learns to combine them (this is the roadmap's "high-level search
strategy" capability). Handlers are thin wrappers over
``sources.search_source``;
they enforce the API-call budget and cross-round query dedup, apply the
active
time scope (incremental mode), and record every call for playbook
harvesting.
"""
from __future__ import annotations
import json
import re
from datetime import datetime, timezone
from typing import Any
from ..analysis import operator_signature, source_query_key, SourceQuerySpec
from ..engine.gaps import SOURCE_CHANNEL
from ..sources import build_arxiv_search_query, search_source
from .context import ToolContext
_NVD_SCHEMA = {
   "type": "object",
   "properties": {
      "query_text": {"type": "string", "description": "Free-text mission
query / keywordSe
arch."},
      "nvd_cwe_id": {"type": "string", "description": "CWE filter, e.g.
CWE-94, CWE-502."}
,
      "nvd_cvss_v3_severity": {"type": "string", "description": "HIGH or
CRITICAL."},
      "nvd_pub_start_date": {"type": "string", "description": "ISO-8601 ext,
e.g. 2026-07-
01T00:00:00.000"},
      "nvd_pub_end_date": {"type": "string", "description": "ISO-8601 ext;
pair with start
."},
      "nvd_has_kev": {"type": "boolean", "description": "Only CVEs in the
CISA KEV catalog
."},
      "nvd_keyword_exact_match": {"type": "boolean", "description": "Phrase
match for mult
i-word keywords."},
      "nvd_cve_id": {"type": "string", "description": "Direct CVE lookup."},
      "nvd_start_index": {"type": "integer", "minimum": 0, "description":
"NVD page offset
."},
      "candidate_id": {"type": "string", "description": "Approved adaptive
candidate ID."}
,
   },
   "required": ["query_text"],
}
_ARXIV_SCHEMA = {
   "type": "object",
   "properties": {
      "query_text": {"type": "string", "description": "Mission query (used
if no arxiv_sea
```

```
rch_query)."},
       "arxiv_search_query": {
          "type": "string",
          "description": 'arXiv query language: all:/ti:/abs:, AND/OR,
quoted phrases, cat
:cs.CR. '
          'E.g. (all:"prompt injection") AND (cat:cs.CR)',
       },
       "arxiv_start": {"type": "integer", "minimum": 0, "description": "arXiv
page offset."
```

```
},
       "candidate_id": {"type": "string", "description": "Approved adaptive
candidate ID."}
,
    },
    "required": ["query_text"],
}
_CISA_SCHEMA = {
    "type": "object",
    "properties": {
       "query_text": {"type": "string", "description": "Keyword to match
against KEV entrie
s."},
       "cisa_keyword": {"type": "string", "description": "Explicit keyword
override."},
       "cisa_cve_ids": {"type": "array", "items": {"type": "string"},
"description": "Speci
fic CVE IDs."},
       "candidate_id": {"type": "string", "description": "Approved adaptive
candidate ID."}
,
    },
    "required": ["query_text"],
}
_OSV_SCHEMA = {
    "type": "object",
    "properties": {
       "query_text": {"type": "string", "description": "Mission query /
identifier."},
       "osv_ecosystem": {"type": "string", "description": "e.g. PyPI, npm,
Go."},
       "osv_package_name": {"type": "string", "description": "Package to
query (pair with e
cosystem)."},
       "osv_purl": {"type": "string", "description": "Package URL."},
       "osv_vuln_id": {"type": "string", "description": "OSV/GHSA/PYSEC/CVE
id for direct l
ookup."},
       "candidate_id": {"type": "string", "description": "Approved adaptive
candidate ID."}
,
    },
    "required": ["query_text"],
}
_PARAM_KEYS: dict[str, tuple[str, ...]] = {
    "nvd_cve_api": (
       "nvd_cwe_id", "nvd_cvss_v3_severity", "nvd_pub_start_date",
"nvd_pub_end_date",
       "nvd_has_kev", "nvd_keyword_exact_match", "nvd_cve_id",
       "nvd_start_index",
    ),
    "arxiv_api": ("arxiv_search_query", "arxiv_start"),
    "cisa_kev_json": ("cisa_keyword", "cisa_cve_ids"),
    "osv_dev_api": ("osv_ecosystem", "osv_package_name", "osv_purl",
"osv_vuln_id"),
```

```
}
def validate_adaptive_candidate_call(
    ctx: ToolContext, source_name: str, candidate_id: str
) -> str | None:
    """Apply the hook/handler shared adaptive-query safety checks."""
    if not ctx.adaptive:
        return None
```

```
    candidate = ctx.approved_candidates.get(candidate_id)
    if candidate is None:
        return "Adaptive source calls require an approved candidate_id."
    if candidate.source_name != source_name:
        return f"Candidate {candidate.candidate_id} is for
{candidate.source_name}."
    if candidate.evidence_channel != SOURCE_CHANNEL[source_name]:
        return "Candidate evidence_channel does not match the selected
source."
    if candidate_exceeds_time_scope(ctx, source_name, candidate.params):
        return "Candidate time operators exceed the authoritative incremental
window."
    params = {
        key: value
        for key, value in candidate.params.items()
        if key in _PARAM_KEYS[source_name] and value not in (None, "", [], {})
    }
    _apply_time_scope(source_name, params, ctx)
    spec = SourceQuerySpec(
        source_name=source_name,
        query_text=candidate.query_text,
        target_topics=list(candidate.target_topics),
        max_results=ctx.max_results_per_call,
        strategy_name="agent",
        params=params,
    )
    if source_query_key(spec) in ctx.executed_keys:
        return "This exact approved query already ran."
    if len(ctx.executed_calls) >= ctx.max_calls_this_round:
        return "Per-round source-call budget exhausted."
    if not ctx.budget_remaining():
        return "API-call budget exhausted for this run."
    return None
def candidate_exceeds_time_scope(
    ctx: ToolContext, source_name: str, params: dict[str, Any]
) -> bool:
    since, until = ctx.source_time_scope(source_name)
    if since is None:
        return False
    if source_name == "nvd_cve_api":
        start = _parse_time_operator(params.get("nvd_pub_start_date"))
        end = _parse_time_operator(params.get("nvd_pub_end_date"))
        return bool((start and start < since) or (until and end and end >
until))
    if source_name == "arxiv_api":
        query = str(params.get("arxiv_search_query") or "")
        match = re.search(
            r"submittedDate:\[(\d{12})\s+TO\s+(\d{12})\]", query,
re.IGNORECASE
        )
        if not match:
            return False
        start = datetime.strptime(match.group(1),
"%Y%m%d%H%M").replace(tzinfo=timezone.utc)
        end = datetime.strptime(match.group(2),
```

```
"%Y%m%d%H%M").replace(tzinfo=timezone.utc)
       return start < since or bool(until and end > until)
```

```
    return False
def _parse_time_operator(value: Any) -> datetime | None:
    if not value:
        return None
    try:
        parsed = datetime.fromisoformat(str(value).replace("Z", "+00:00"))
    except ValueError:
        return None
    return parsed.replace(tzinfo=parsed.tzinfo or
timezone.utc).astimezone(timezone.utc)
def _apply_time_scope(source_name: str, params: dict[str, Any], ctx:
ToolContext) -> None:
    """Inject native date operators for sources that support them
(incremental mode)."""
    since, until = ctx.source_time_scope(source_name)
    if since is None:
        return
    if source_name == "nvd_cve_api" and (ctx.adaptive or "nvd_pub_start_date"
not in params)
:
        params["nvd_pub_start_date"] = since.strftime("%Y-%m-%dT%H:%M:%S.000")
        end = until if until is not None else datetime.now(timezone.utc)
        params["nvd_pub_end_date"] = end.strftime("%Y-%m-%dT%H:%M:%S.000")
    elif source_name == "arxiv_api":
        existing = params.get("arxiv_search_query") or ""
        if ctx.adaptive and "submittedDate" in existing:
            existing = re.sub(
                r"\s+AND\s+submittedDate:\[[^\]]+\]",
                "",
                str(existing),
                flags=re.IGNORECASE,
            ).strip()
        if "submittedDate" not in existing:
            frm = since.strftime("%Y%m%d%H%M")
            to = until.strftime("%Y%m%d%H%M") if until is not None else
since.strftime("%Y%m
%d") + "2359"
            window = f"submittedDate:[{frm} TO {to}]"
            params["arxiv_search_query"] = f"({existing}) AND {window}" if
existing else win
dow
def _run_call(ctx: ToolContext, source_name: str, args: dict[str, Any]) ->
dict[str, Any]:
    query_text = str(args.get("query_text") or "").strip()
    candidate_id = str(args.get("candidate_id") or "").strip() or None
    params = {
        key: args[key]
        for key in _PARAM_KEYS[source_name]
        if key in args and args[key] not in (None, "", [], {})
    }
    query_intent = "gap_fill"
    target_topics = list(ctx.target_topics)
    target_gap_ids: list[str] = []
    evidence_channel = SOURCE_CHANNEL[source_name]
    if ctx.adaptive:
```

```
       denial = validate_adaptive_candidate_call(ctx, source_name,
candidate_id or "")
       if denial:
```

```
            return _text(f"Denied: {denial}")
        candidate = ctx.approved_candidates[candidate_id or ""]
        query_text = candidate.query_text
        params = {
            key: value
            for key, value in candidate.params.items()
            if key in _PARAM_KEYS[source_name] and value not in (None, "", [],
{})
        }
        query_intent = candidate.query_intent
        target_topics = list(candidate.target_topics)
        target_gap_ids = list(candidate.target_gap_ids)
    if source_name == "arxiv_api" and not params.get("arxiv_search_query"):
        params["arxiv_search_query"] = build_arxiv_search_query(
            query_text, target_topics or ctx.target_topics
        )
    _apply_time_scope(source_name, params, ctx)
    spec = SourceQuerySpec(
        source_name=source_name, query_text=query_text,
target_topics=target_topics,
        max_results=ctx.max_results_per_call, strategy_name="agent",
params=params,
    )
    key = source_query_key(spec)
    if key in ctx.executed_keys:
        return _text(f"Skipped: this exact {source_name} query already ran.
Vary the operato
rs or query text.")
    if ctx.adaptive and len(ctx.executed_calls) >= ctx.max_calls_this_round:
        return _text("Skipped: per-round source-call budget exhausted. Call
submit_round_sum
mary.")
    if not ctx.budget_remaining():
        return _text("Skipped: API-call budget exhausted for this run. Call
submit_round_sum
mary.")
    items, stat = search_source(
        source_name, query_text or " ".join(ctx.target_topics[:3]),
        target_topics=target_topics or ctx.target_topics,
        max_results=ctx.max_results_per_call,
        **params,
    )
    ctx.executed_keys.add(key)
    ctx.api_calls_used += 1
    ctx.stats.append(stat)
    new_ids: list[str] = []
    kept = 0
    duplicates = 0
    for item in items:
        if not ctx.in_time_scope(item):
            continue
        kept += 1
        if item.item_id in ctx.existing_item_ids or item.item_id in
ctx.collected_items:
            duplicates += 1
```

```
            continue
        item.metadata["collection_strategy"] = "agent"
```

```
        item.metadata["source_query"] = query_text
        item.metadata["source_query_params"] = params
        ctx.collected_items[item.item_id] = item
        new_ids.append(item.item_id)
    has_more = source_name in {"nvd_cve_api", "arxiv_api"} and len(items) >=
ctx.max_results
_per_call
    call_id = f"{ctx.run_id}:call:{ctx.api_calls_used}"
    ctx.executed_calls.append(
        {
            "call_id": call_id,
            "candidate_id": candidate_id,
            "source_name": source_name,
            "query_text": query_text,
            "params": params,
            "query_intent": query_intent,
            "evidence_channel": evidence_channel,
            "target_topics": target_topics,
            "target_gap_ids": target_gap_ids,
            "new_item_ids": new_ids,
            "signature": operator_signature(source_name, params),
            "raw_count": len(items),
            "in_scope_count": kept,
            "duplicate_count": duplicates,
            "has_more": has_more,
            "success": stat.success,
            "error_type": stat.error_type,
            "latency_ms": stat.latency_ms,
        }
    )
    if candidate_id:
        ctx.approved_candidates.pop(candidate_id, None)
    if not stat.success:
        return _text(f"{source_name} error: {stat.error_type}. Try a different
source or ope
rators.")
    titles = "; ".join(item.title[:70] for item in
list(ctx.collected_items.values())[-3:])
    return _text(
        f"{source_name}: fetched {len(items)} raw / {kept} in-scope, "
        f"{duplicates} duplicate, {len(new_ids)} new, has_more={has_more}. "
        f"operators={_fmt(params)}. Newest: {titles or '(none)'}"
    )
def build_source_tools(ctx: ToolContext) -> list[Any]:
    from claude_agent_sdk import tool
    @tool("search_nvd", "Search the NVD CVE API 2.0 for LLM/AI-related CVEs.
"
          "Combine keyword_text with nvd_cwe_id / nvd_cvss_v3_severity / date
window / nvd_h
as_kev "
          "to cut noise (e.g. CWE-502 unsafe deserialization, CWE-94 code
injection).", _NVD
_SCHEMA)
    async def search_nvd(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "nvd_cve_api", args)
```

```
   @tool("search_arxiv", "Search arXiv for LLM-security papers. Prefer
arxiv_search_query w
```

```
ith the "
        "arXiv query language (field prefixes, AND/OR, quoted phrases,
cat:cs.CR) for prec
ision.",
        _ARXIV_SCHEMA)
    async def search_arxiv(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "arxiv_api", args)
    @tool("search_cisa_kev", "Search the CISA Known Exploited Vulnerabilities
catalog by key
word or "
        "specific CVE IDs. Best for confirming real-world exploited AI-stack
CVEs.", _CISA
_SCHEMA)
    async def search_cisa_kev(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "cisa_kev_json", args)
    @tool("search_osv", "Search OSV.dev for open-source ecosystem advisories.
Use osv_ecosys
tem + "
        "osv_package_name (e.g. PyPI/mlflow) or osv_vuln_id for direct
lookup.", _OSV_SCHE
MA)
    async def search_osv(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "osv_dev_api", args)
    return [search_nvd, search_arxiv, search_cisa_kev, search_osv]
def _fmt(params: dict[str, Any]) -> str:
    return json.dumps(params, ensure_ascii=False) if params else "base"
def _text(message: str) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": message}]}
#   backend/intel_agent/topics.py
"""LLM-security topics — re-export shared taxonomy from
``common.topics``."""
from __future__ import annotations
from common.topics import (  # noqa: F401
    ALL_SECURITY_TOPICS,
    CORE_SECURITY_TOPICS,
    EXTENDED_SECURITY_TOPICS,
    TARGET_SECURITY_TOPICS,
    TOPIC_ANCHORS,
    TOPIC_KEYWORDS,
    build_gap_query,
    detect_topics,
    is_core_topic,
    tokenize,
)
__all__ = [
    "ALL_SECURITY_TOPICS",
    "CORE_SECURITY_TOPICS",
    "EXTENDED_SECURITY_TOPICS",
    "TARGET_SECURITY_TOPICS",
    "TOPIC_ANCHORS",
    "TOPIC_KEYWORDS",
    "build_gap_query",
    "detect_topics",
    "is_core_topic",
    "tokenize",
```

]